

Modélisation Orientée Objet avec UML

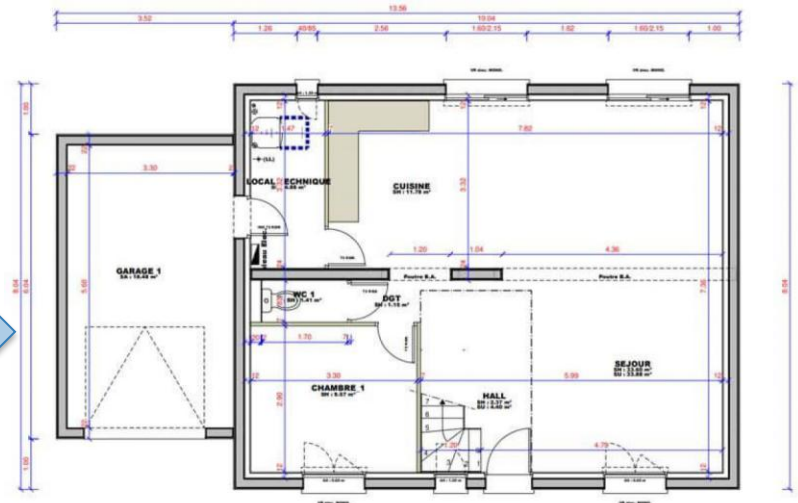
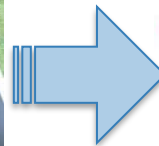
« *Le sage réfléchit avant d'agir* »

Proverbe persan

Mise en situation

Construire une maison:

- Avant de commencer à construire, il faut comprendre ce que le propriétaire désire => recensement des besoins client (nombre de chambres, emplacement des chambres, jardin..)
- Demander plus de détails (questions, réunions etc.) pour aller vers une modélisation plus technique: mesures exactes, équipements et qualité du matériel, finition, etc.



Mise en situation

- Développement logiciel: Un grand chantier
 - Définir : Fonctionnalités principales, interfaces, charte graphique, web ou mobile ou desktop, navigation dans l'application, les accès, type d'utilisateurs, etc.
- Comprendre le besoin du client
- Définir l'architecture du système
- Planifier et estimer le coût du projet
- Allouer ressources et compétences nécessaires
- Définir la Qualité attendue
- Définir les Exigences légales
- Préparer l'Environnement de développement / test
- Compatibilité et intégration
- Accès
- ...



Mise en situation

- Développement logiciel: Un grand chantier
 - ▷ Différents rôles dans le software engineering:
 - ▷ Expert du domaine/ Client (Product owner)
 - ▷ Chef de projet
 - ▷ **Business Analyst**
 - ▷ **Software architect : comprendre et formaliser le besoin du client.**
 - ▷ Développeur / ingénieur
 - ▷ Code, base de données, sécurité, etc
 - ▷ Testeur
 - ▷ Administrateur
 - ▷ Exploitation
 - ▷ ...

Mise en situation

■ Développement logiciel: Un grand chantier

- ▷ D'où la nécessité de bien comprendre le besoin client en termes de fonctionnalités et de qualité attendues.
- ▷ Bien comprendre les contraintes, difficultés et complexité du projet.
- ▷ D'où la nécessité d'une analyse approfondie du projet et de son cahier des charges.

Plan de ce cours

- Génie logiciel
- Historique des méthodes de conception
- Genèse du langage UML
- La norme UML
- Les diagrammes UML
- Diagramme des cas d'utilisation
- Diagramme de séquences
- Diagramme de classes
- Diagramme d'état transition
- Diagramme d'activité
- Diagramme de déploiement

Génie Logiciel

Introduction

- Les logiciels, suivant leur taille, peuvent être développés par une seule personne, une petite équipe, ou un ensemble d'équipes coordonnées (et/ou décentralisées)
- Le développement de grands logiciels par de grandes équipes peut poser d'importants problèmes de conception et de coordination, entre les équipes et avec le client final.
- Ce qui engendre :
 - ▷ Dépassement des coûts et des délais (pénalités).
 - ▷ Les difficultés de maintenance et d'évolution
 - ▷ Non fiabilité (traitement et performance)
 - ▷ Non respect des spécifications : Projets ne répondant pas aux besoins du client (projets abandonnés).
 - ▷ Coûts financiers, perte de clients, l'image de l'entreprise, etc.

Introduction

■ En chiffres :

- ▷ 66% des projets technologiques se terminent par un échec partiel ou total.
- ▷ Moins de 10 % des grands projets réussissent.
- ▷ 1/10 pour les petits échouent.
- ▷ 31 % des projets informatiques américains ont été purement et simplement annulés.
- ▷ Des milliards d'investissement dans des projets de digitalisation perdus.
- ▷ ...

- According to the Standish Group's Annual CHAOS 2020 report, 66% of technology projects (based on the analysis of 50,000 projects globally) end in partial or total failure. While larger projects are more prone to encountering challenges or failing altogether, even the smallest software projects fail one in ten times. Large projects are successful less than 10% of the time.
- Standish also found that 31% of US IT projects were canceled outright and the performance of 53% 'was so worrying that they were challenged.'
- Research from McKinsey in 2020 found that 17% of large IT projects go so badly, they threaten the very existence of the company.
- BCG (2020) estimated that 70% of digital transformation efforts fall short of meeting targets. A 2020 CISQ report found the total cost of unsuccessful development projects among US firms is an estimated \$260B, while the total cost of operational failures caused by poor quality software is estimated at \$1.56 trillion.

Illustration

■ Chacun sa perception !



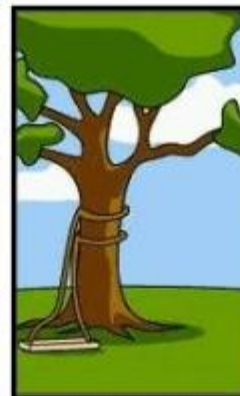
Comment le client a exprimé son besoin



Comment le chef de projet l'a compris



Comment l'ingénieur l'a conçu



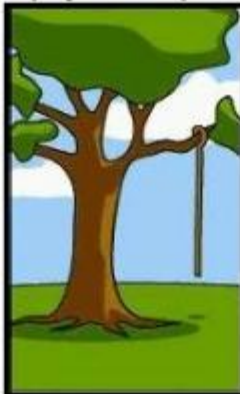
Comment le programmeur l'a écrit



Comment le responsable des ventes l'a décrit



Comment le projet a été documenté



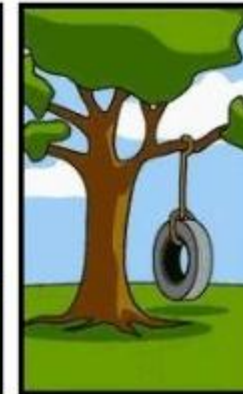
Ce qui a finalement été installé



Comment le client a été facturé



Comment la hotline répond aux demandes



Ce dont le client avait réellement besoin

Génie logiciel

- Le génie logiciel (software engineering) représente l'application de principes d'ingénierie au domaine de la création de logiciels. Il consiste à identifier et à utiliser des méthodes, des pratiques et des outils permettant de maximiser les chances de réussite d'un projet logiciel:
 - ▷ Méthodes de gestion de projets
 - ▷ Méthodes de conception
 - ▷ Méthodes de développement et d'intégration
 - ▷ Langages de programmation
 - ▷ Paradigme de programmation (et bonnes pratiques)
 - ▷ ...

« Produire des logiciels de qualité »

Génie logiciel

■ On peut mesurer la qualité d'un logiciel à travers plusieurs critères dont:

1. **L'exactitude** : aptitude d'un programme à fournir le résultat voulu et à répondre ainsi aux spécifications.
2. **La robustesse** : aptitude à bien réagir lorsque l'on s'écarte des conditions normales d'utilisation.
3. **L'extensibilité** (sustainability): facilité avec laquelle un programme pourra être adapté pour répondre à l'évolution des spécifications
4. **La réutilisabilité** : possibilité d'utiliser certaines parties du logiciel pour résoudre un autre problème.
5. **La portabilité** : facilité avec laquelle on peut exploité un même logiciel dans différentes implémentations.
6. **L'efficience** : temps d'exécution, taille mémoire.

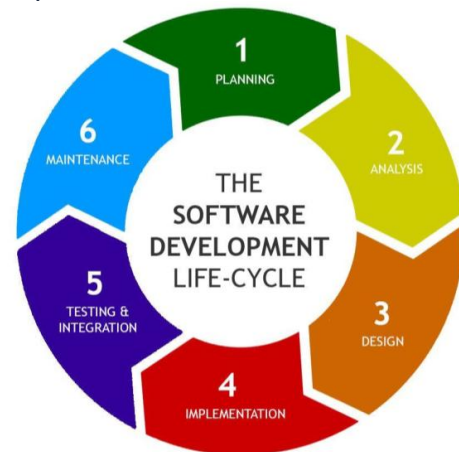
Cycle de vie logiciel

- **SDLC (Software Development Life Cycle):** est un processus structuré que les équipes de développement utilisent pour concevoir, développer, tester et déployer des logiciels de haute qualité. L'objectif est de produire un logiciel qui répond ou dépasse les attentes des clients, tout en respectant les délais et le budget alloués.
- **Planification et Analyse des Besoins :** C'est la première étape, où les parties prenantes (clients, chefs de projet, experts du domaine) définissent les exigences du projet. On y détermine: les objectifs, le coût-bénéfice, la portée/périmètre, la planification initiale, l'estimation et l'affectation des ressources.
- **Conception (Design):** Une fois les exigences comprises, les architectes logiciels et les développeurs créent les plans du système. Cette phase définit l'architecture globale du logiciel, les composants, les interfaces, les bases de données et les aspects de sécurité. On produit souvent des documents de conception (spécifications techniques) qui guideront les développeurs.



Cycle de vie logiciel

- **Développement (Implementation):** C'est la phase où les développeurs écrivent le code source du logiciel en suivant les spécifications de la phase de conception. Ils choisissent un langage de programmation (comme Python, Java, C++) et utilisent divers outils pour construire les fonctionnalités du programme.
- **Tests :** Une fois le code écrit, il doit être testé de manière approfondie pour s'assurer qu'il est exempt de défauts (bugs) et qu'il répond bien aux exigences initiales.
- **Déploiement:** ès avoir passé les tests avec succès, le logiciel est prêt à être mis à la disposition des utilisateurs (MEP). Cette phase peut être simple, comme le téléchargement d'une application sur un site web, ou complexe, impliquant l'installation sur de multiples serveurs.
- **Maintenance :** Une fois le logiciel en production, le cycle de vie n'est pas terminé. La phase de maintenance commence. Elle consiste à surveiller le logiciel pour s'assurer qu'il fonctionne correctement, à corriger les bugs découverts par les utilisateurs, et à apporter des mises à jour ou des améliorations au fil du temps.



Paradigmes de programmation

■ Programmation procédurale

- ▷ Les données et traitements sont séparés. Les données traitées via des variables et les fonctions qui opèrent sur les données passées comme arguments.
- ▷ Aucun lien sémantique entre données et fonctions.
- ▷ Aucune séparation entre données (propriétés à manipuler) et variables locales.
- ▷ Difficile à maintenir. En cas de modification, il est difficile de repérer les endroits du code qui seront affectés par cette modification.

```
struct Etudiant {  
    int id;  
    char nom[50];  
    float moyenne;  
};  
void afficherEtudiant(struct Etudiant e) {  
    printf("ID: %d\n", e.id);  
    printf("Nom: %s\n", e.nom);  
    printf("Moyenne: %.2f\n", e.moyenne);  
}
```

Approche orientée objet

- **Programmation orientée objet (POO)** est un paradigme de programmation qui considère le logiciel comme une collection d'objets dissociés, identifiés et possédant des caractéristiques. Une caractéristique est soit une donnée de l'objet, soit un traitement appliqué sur l'objet.

Approche orientée objet

■ Objectif de la POO:

- ▷ Établir le lien entre données et traitements au niveau de l'implémentation.
- ▷ En plus de:
 - Réutilisabilité
 - Maintenabilité
 - Lisibilité
 - Robustesse & sécurité (face aux changements et erreurs de manipulation)
 - Modularité
 - Une représentation du monde réel
 - Facile à utiliser pour des programmes ou applications complexes
 - ...

Méthode de conception et d'analyse

Méthodes de conception et d'analyse

- Besoin d'un langage, une formalisation, une documentation.
- Un moyen fiable de communication sans aucune ambiguïté qui décrit le fonctionnement de l'application, les règles de gestion, les spécifications etc.
- Un moyen rapide (non des documents de centaines de pages!!).
- Un moyen graphique serait le plus idéal.
- Besoin aussi de processus, de démarche.

Méthodes de conception et d'analyse

- **Merise** : séparation entre données et traitements.
- Démarche complète : méthode + langage
- Approche globale qui traite données et traitements via différents modèles.
- Définit plusieurs niveaux d'abstraction (physique, logique, conceptuel).
- Influencée par le SGBDR.
- Pas de fusion possible entre les deux aspects: données, traitements.

Méthode orientée objet

Besoin d'une méthode qui:

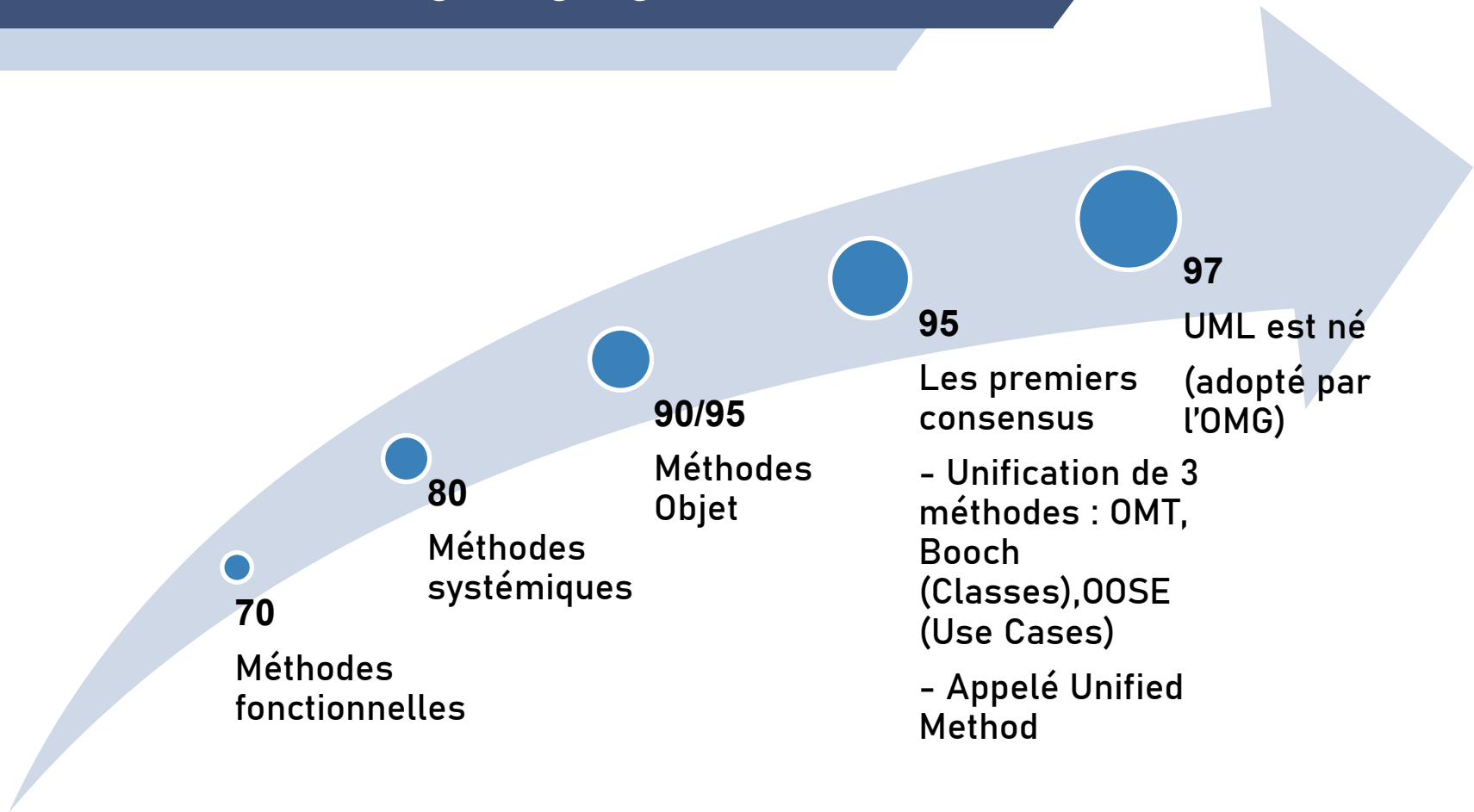
- S'aligne avec les concepts de la programmation OO: classe, héritage, abstraction etc.
 - ▷ Passage conception/implémentation facilité (conceptuel / physique)
- Intègre données et traitements.
- Profite des avantages de l'orienté objet: la réutilisabilité des composants.
 - ▷ Améliorer la productivité.

Méthode orientée objet

- Années 80/90: Apparition d'une cinquantaine de méthodes OO.
- Besoin de combiner plusieurs méthodes pour faire la conception: chaque méthode traite un volet du projet.
- Les développeurs n'ont toujours pas de moyen de communication commun, car chacun utilise une méthode différente.
 - ▷ Vocabulaire différent: risque d'ambiguïtés, d'incompréhensions.
- L'absence de consensus sur une méthode d'analyse objet a longtemps freiné l'essor des technologies objet.

La genèse d'UML

Unified Modeling Language



L'Object Management Group (OMG) est un consortium international à but non lucratif créé en 1989 dont l'objectif est de standardiser et promouvoir le modèle objet sous toutes ses formes.

Le langage UML

- UML est une **norme** OMG (Object Management Group).
- UML est un **langage** de modélisation objet:
 - ▷ Langage indépendant des langages de développement et de leurs contraintes techniques spécifiques.
 - ▷ UML est un langage formel: décrit de manière précise les concepts objet.
 - ▷ Contient les éléments requis dans un langage: des concepts, une syntaxe et une sémantique.
 - ▷ Compréhensible par l'homme et la machine.
- UML est un support de **communication**:
 - ▷ La représentation et la compréhension (sans ambiguïtés) de solutions objet.
 - ▷ Sa notation **graphique** permet d'exprimer visuellement une solution objet.
 - ▷ Langage commun et indépendant des langages de programmation.

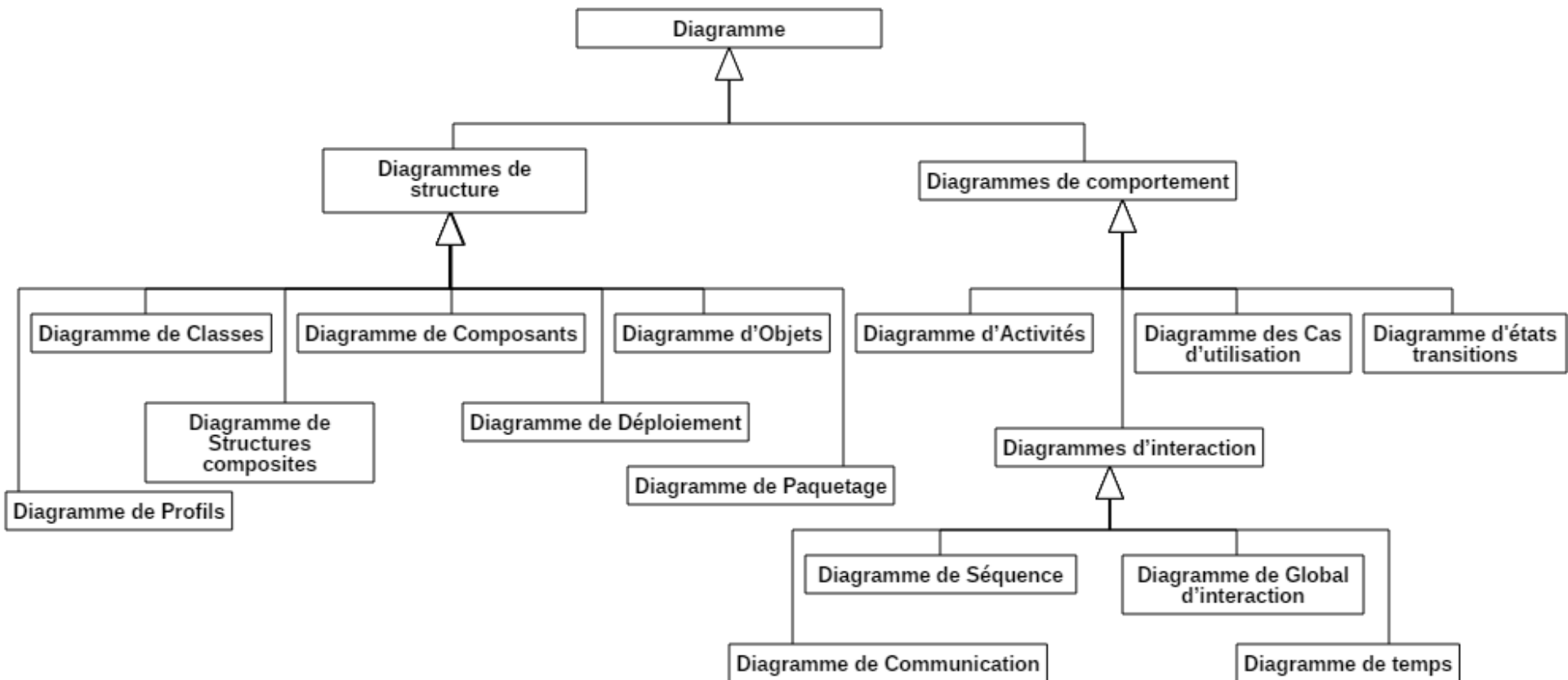
Le langage UML

- UML est un **cadre méthodologique**
 - ▷ Offre une panoplie de **diagrammes** qui permettent de représenter différents aspects d'une solution.
 - ▷ Combiner différents types de diagrammes UML offre une vue complète des aspects statiques et dynamiques d'un système.
- UML **n'est** pas une méthode
 - ▷ Il ne définit pas le processus d'élaboration des modèles.
 - ▷ Définir un processus dans UML limitera UML à des domaines d'activité précis.
 - ▷ Améliorer un processus est une discipline à part entière.

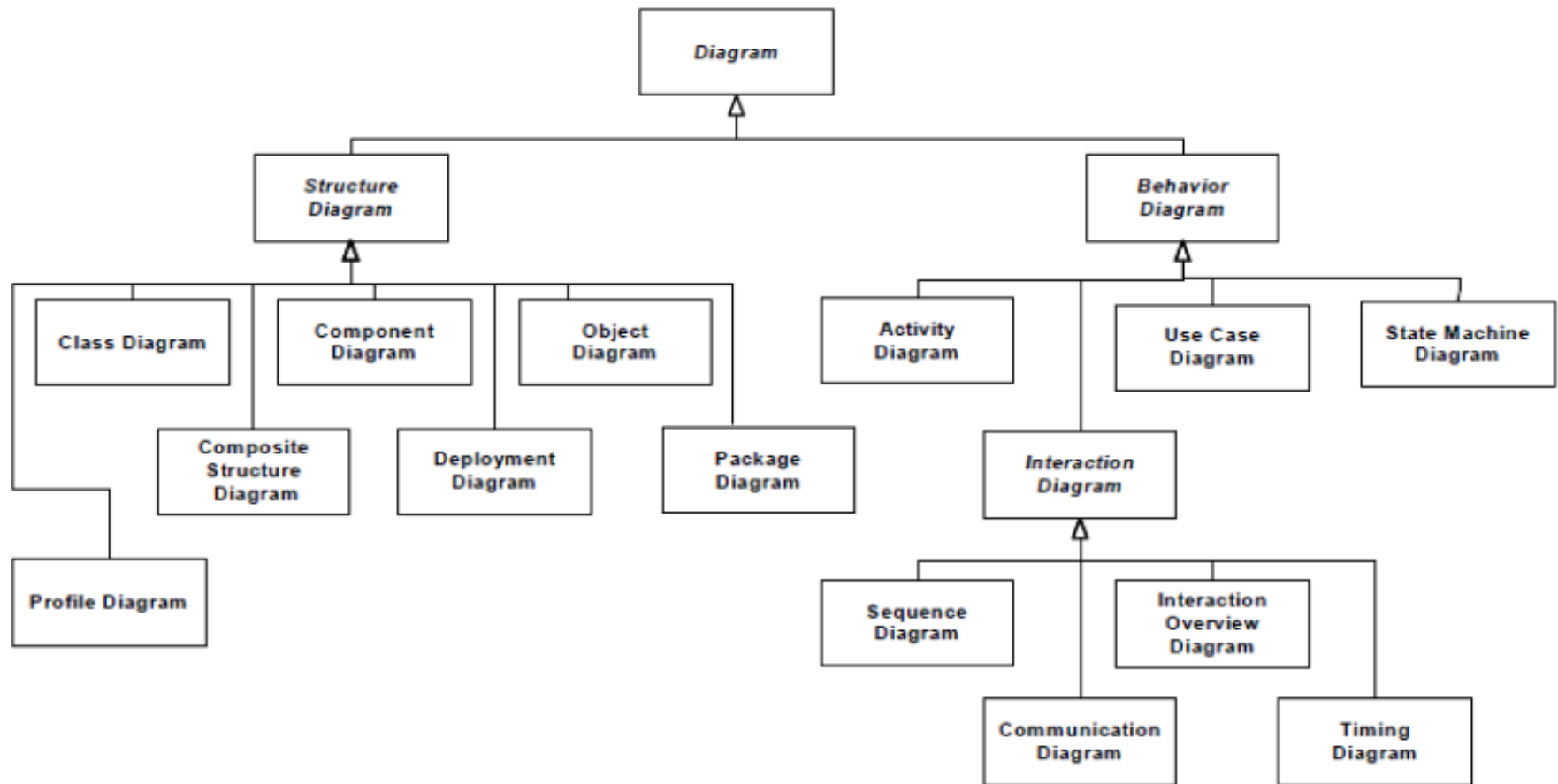
Diagrammes d'UML

- UML 1.1 à 1.4 :
 - ▷ 1997 à 2005. plusieurs améliorations ont été apportées aux 9 diagrammes proposés en termes de sémantiques, de profils et de notation.
- UML 2.0 :
 - ▷ Août 2005.
 - ▷ 13 diagrammes.
 - ▷ Ajout des diagrammes : diag. de paquetages, de structures composites, de temps et de global d'interaction .
- UML 2.5.1 :
 - ▷ Décembre 2017.
 - ▷ 14 diagrammes.
 - ▷ Ajout du diagramme de profils.

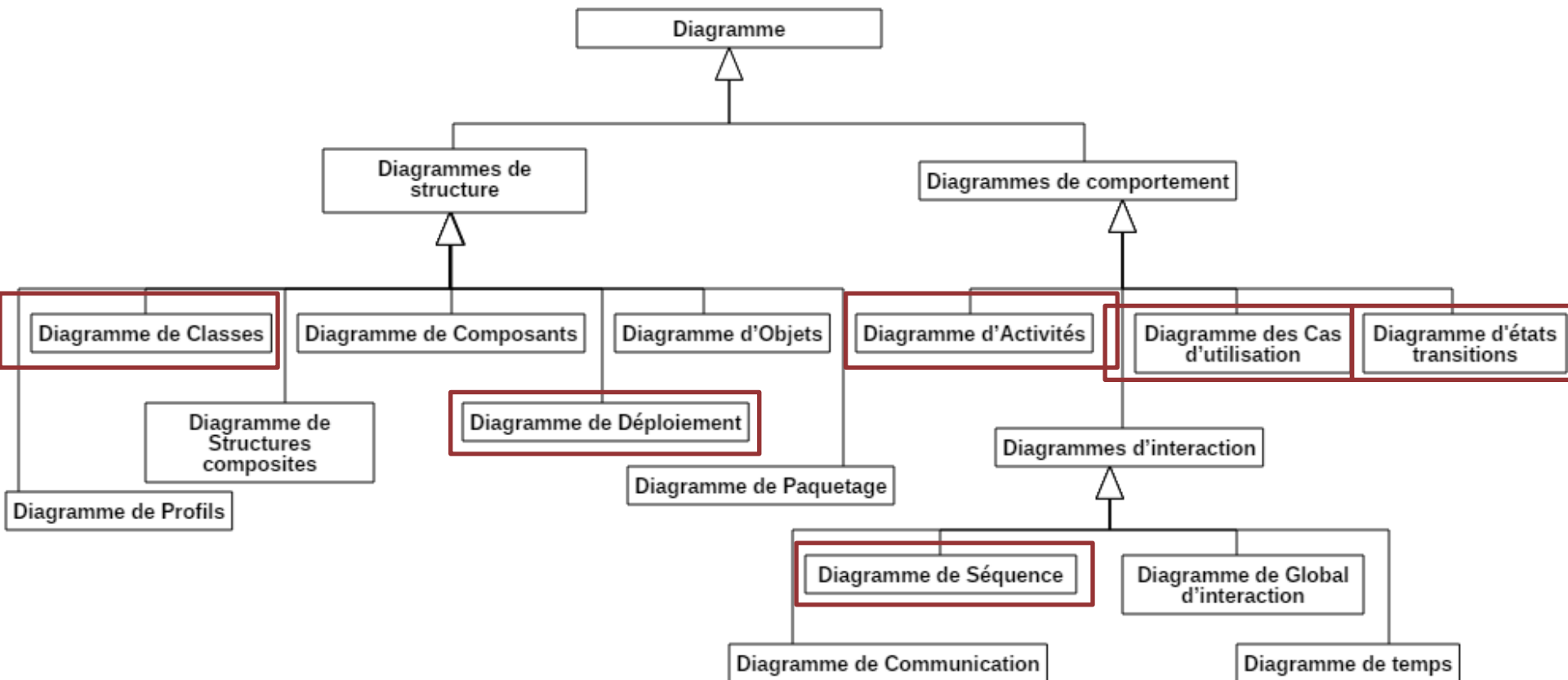
Diagrammes d'UML



Diagrammes d'UML



Diagrammes d'UML

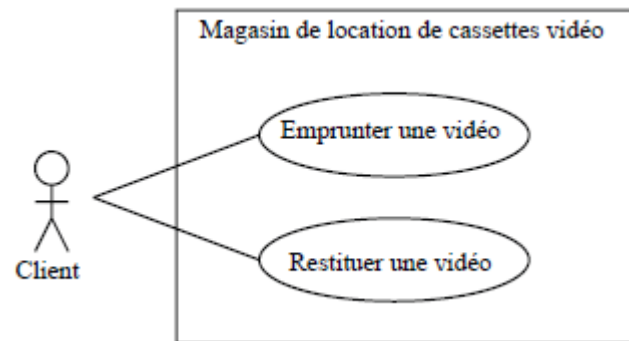


Diagrammes d'UML

1. Diagramme de cas d'utilisation (Use case diagram):

Un diagramme de cas d'utilisation capture le comportement d'un système, d'un sous-système, d'une classe ou d'un composant tel qu'un utilisateur extérieur le voit. Il scinde la fonctionnalité du système en unités cohérentes, les cas d'utilisation, ayant un sens pour les acteurs.

Modélise **à QUOI** sert le système, en organisant les interactions possibles avec les acteurs.

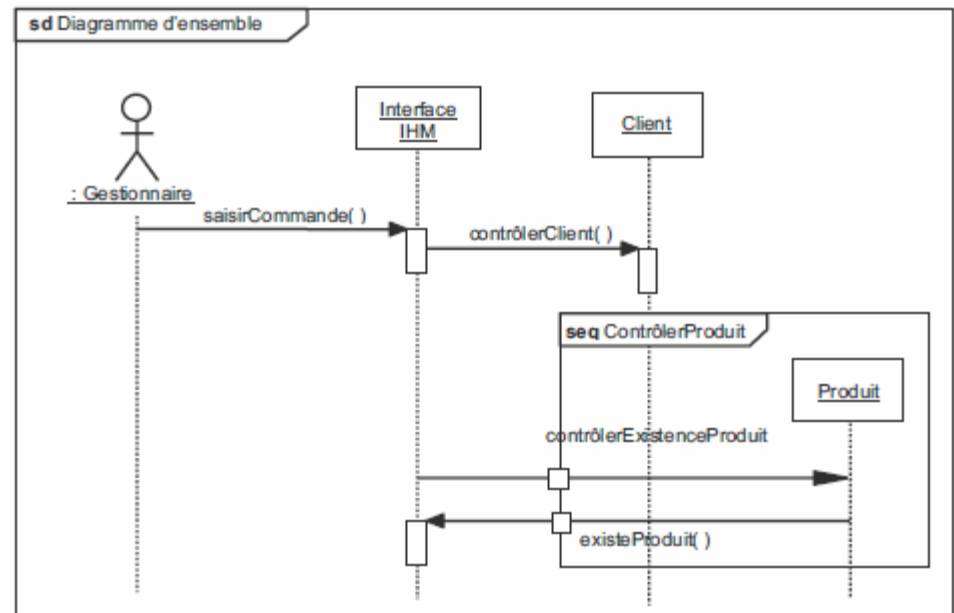


Diagrammes d'UML

2. Diagramme de séquence (Sequence diagram): représente les interactions entre objets en indiquant la chronologie des échanges.

- Cette représentation peut se réaliser par cas d'utilisation en considérant les différents scénarios associés.

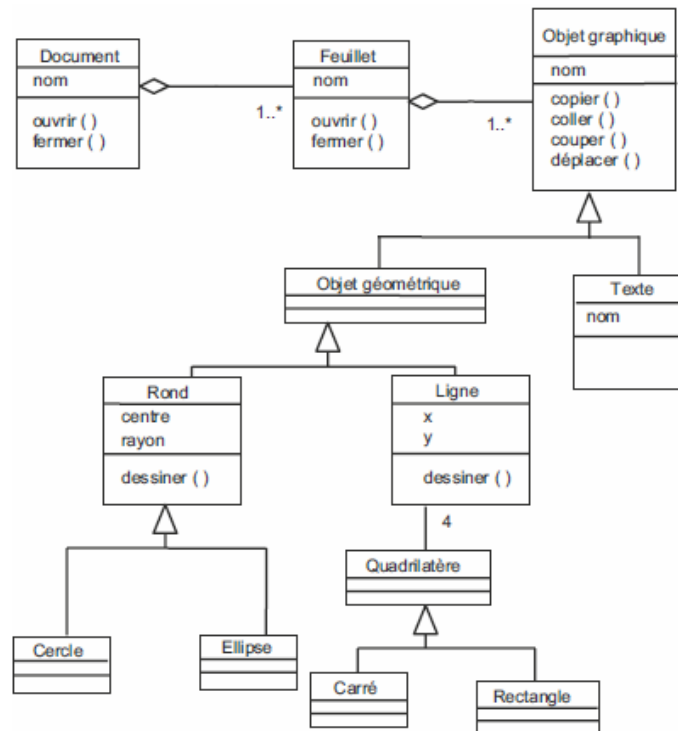
Décrit **COMMENT**
les éléments du système
interagissent entre eux et avec
les acteurs.



Diagrammes d'UML

3 . Diagramme de classes (Class diagram):

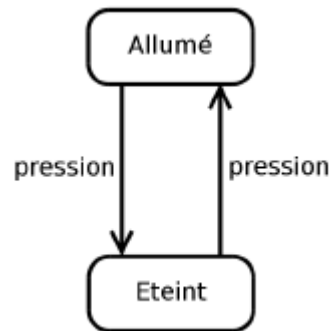
est considéré comme le plus important et le plus utilisé des diagrammes UML. Il décrit les classes d'une application, leurs attributs et méthodes ainsi que les relations (associations) entre les classes.



Diagrammes d'UML

4. Diagramme d'états-transitions (State machine diagram):

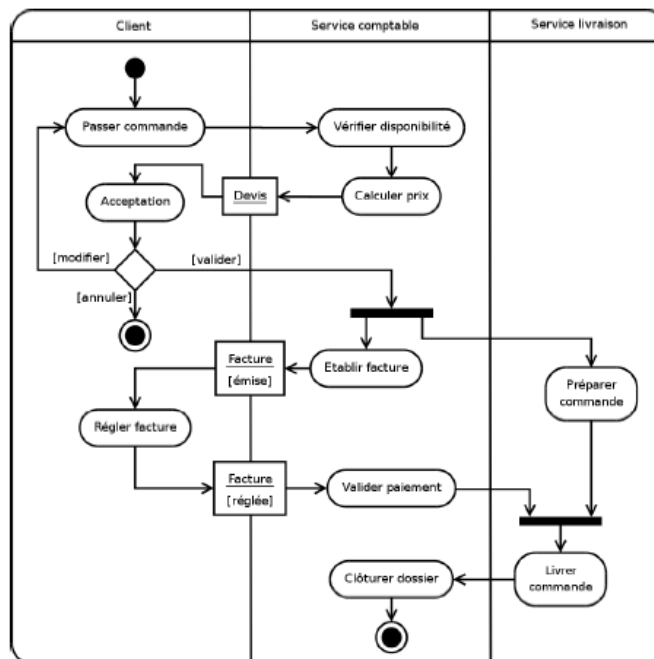
- Décrit le comportement interne d'un objet à l'aide d'un automate à états finis. Ils présentent les séquences possibles d'états et d'actions qu'une instance de classe peut traiter au cours de son cycle de vie en réaction à des événements discrets (de type signaux, invocations de méthode).
- Il spécifie habituellement le comportement d'une instance.



Diagrammes d'UML

5 . Diagramme d'activités (Activity diagram):

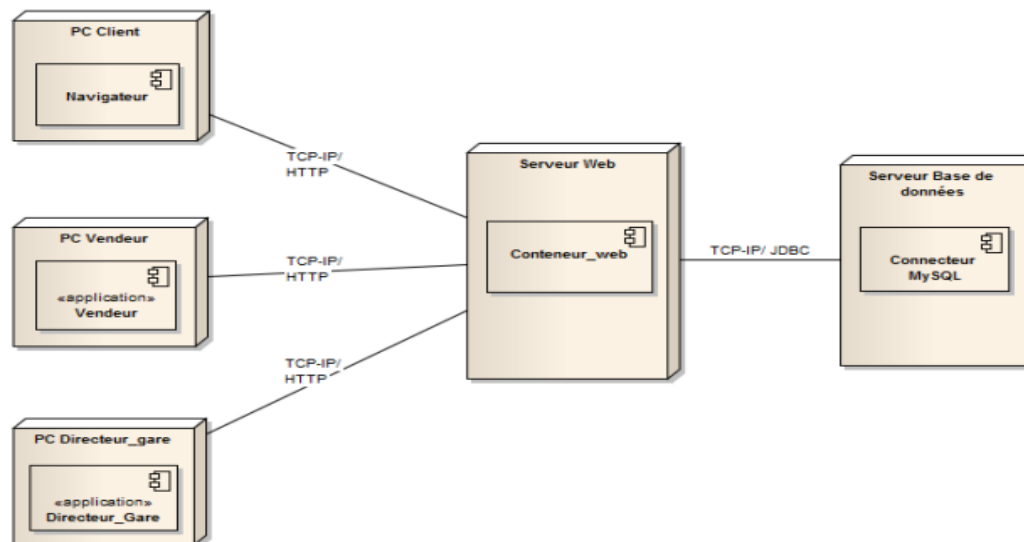
- Permet de mettre l'accent sur les traitements. Particulièrement adapté à la modélisation des workflows et dataflows.
- permet de représenter graphiquement le comportement d'une méthode ou le déroulement d'un cas d'utilisation ou d'un processus.



Diagrammes d'UML

6 . Diagramme de déploiement (Deployment diagram):

- ▷ décrit l'architecture physique ainsi que les relations entre les composants logiciels et matériels d'une application.
- ▷ L'environnement d'exécution ou les ressources matérielles sont appelés « noeuds ». Les parties d'un système qui s'exécutent sur un noeud sont appelées « artefacts ».



Diagrammes d'UML

- Un diagramme UML est une représentation graphique, qui s'intéresse à un aspect précis du modèle.
- C'est une perspective du modèle, pas "le modèle".
- Combinés, les différents types de diagrammes UML offrent une vue complète des aspects statiques et dynamiques d'un système.

'Depuis des temps très anciens, les hommes ont cherché un langage à la fois universel et synthétique, et leurs recherches les ont amenés à découvrir des images, des symboles qui expriment, en les réduisant à l'essentiel, les réalités les plus riches et les plus complexes.

Les images, les symboles parlent, ils ont un langage".

O.M. Aïvanhov (philosophe)

Modélisation avec UML

- Modèle: Un modèle est une abstraction de la réalité.
 - ▷ Une représentation subjective (pertinente) et partielle de la réalité.
 - ▷ Faciliter la compréhension du système et réduire la complexité.
 - ▷ Représenter voire simuler un système.

“A est un bon modèle de B si A permet de répondre de façon satisfaisante à des questions prédéfinies sur B”.

Douglas. T. Ross

(Informaticien américain connu pour ses contributions à la recherche sur les réseaux informatiques)

Diagrammes d'UML

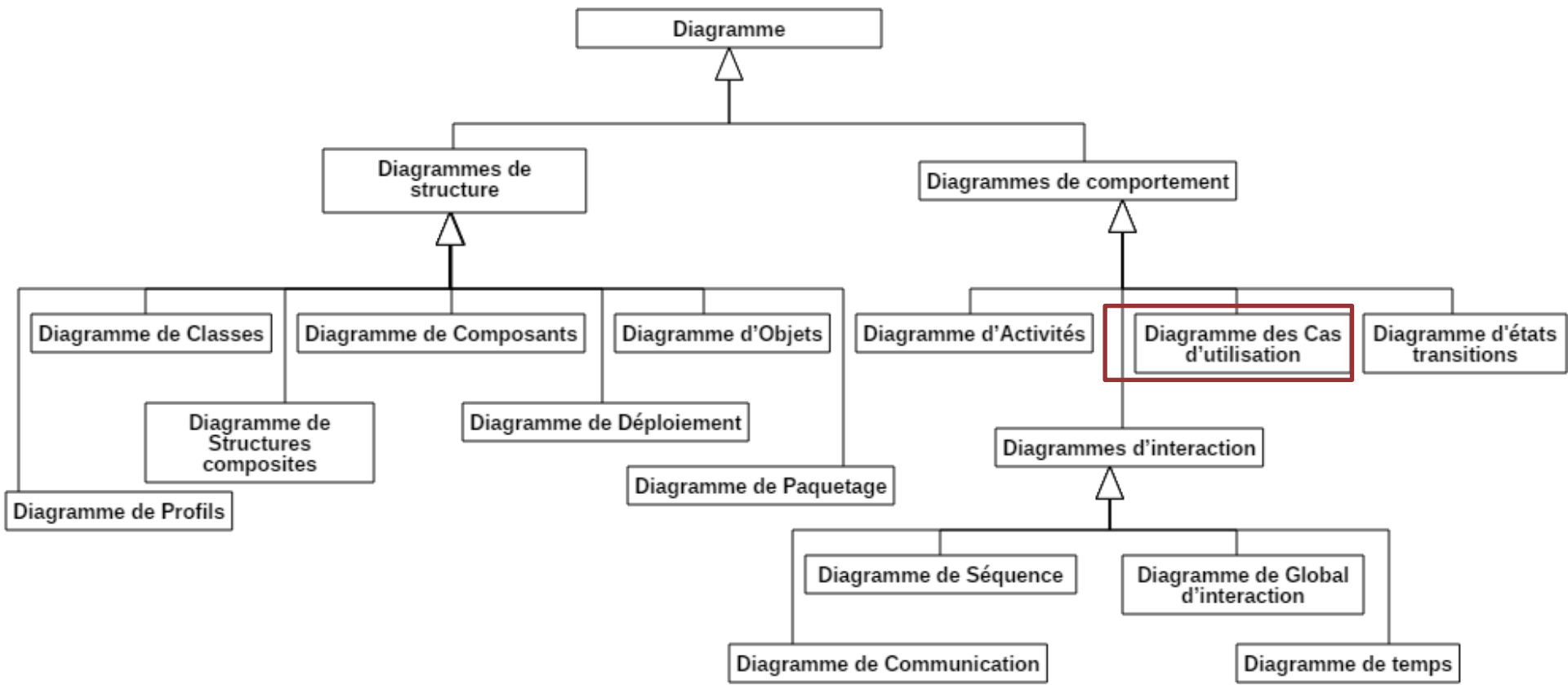


Diagramme de cas d'utilisation

Diagramme des cas d'utilisation

- Les cas d'utilisation constituent un moyen de recueillir et de décrire les **besoins** des acteurs du système (le **QUOI**)
- Le diagramme des cas d'utilisation est un moyen de représentation **graphique** des grandes **fonctionnalités** attendues du système, **compréhensible**, **simple** et **lisible** par les utilisateurs (Maîtrise d'ouvrage).
- Il s'agit de la **première** étape d'analyse d'un système.
- Les cas d'utilisation sont déduits du cahier des spécifications fonctionnelles ainsi que des entretiens (réunions) explicitant ces dernières.
- Les UCs permettent de recenser et représenter le besoin et de le planifier.
- Principaux éléments d'un cas d'utilisation:
 - Acteur : utilisateur du système
 - Cas d'utilisation : fonctionnalité offerte par le système
 - Association : relation entre acteurs, systèmes et cas d'utilisation.

Diagramme des cas d'utilisation

■ Acteur:

- ▶ Est un **utilisateur type (rôle)** qui a toujours le même comportement vis-à-vis d'un cas d'utilisation.
- ▶ les utilisateurs d'un système appartiennent à une ou plusieurs classes d'acteurs selon les rôles qu'ils tiennent par rapport au système.
 - ▶ Une même personne physique peut se comporter autant d'acteurs différents que le nombre de rôles qu'elle joue vis-à-vis du système.
- ▶ Peut être un **humain ou un autre système**.
- ▶ Formalisme:

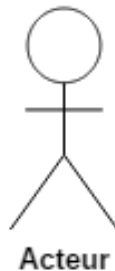
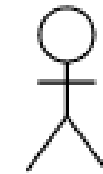


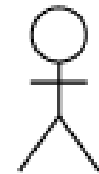
Diagramme des cas d'utilisation

Exemples:

- Application de gestion des ventes
- Site de la FSTM
- Administrateur de l'application (accès, mises à jour etc.)



Client



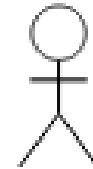
Vendeur



Etudiant



Professeur



Administrateur

Diagramme des cas d'utilisation

■ Cas d'utilisation:

- ▶ Représente une fonctionnalité, service, rendue par le système à ses utilisateurs.
- ▶ À chaque acteur correspond un certain nombre de cas d'utilisation.
- ▶ Un cas d'utilisation modélise donc un service rendu par le système, **sans imposer le mode de réalisation de ce service.**
- ▶ Formalisme:



Diagramme des cas d'utilisation

- **Acteur principal:** est un utilisateur principal, dit actif, du système. Est aussi l'initiateur du cas d'utilisation.
- **Acteur secondaire:** est un acteur externe qui fournit un service au système en cours de conception (ou permet au système d'accomplir ses tâches).
- L'acteur secondaire est généralement précisé dans le DCU à titre d'information.
- Notons que cette distinction n'est pas définie par l'OMG.

Diagramme des cas d'utilisation

■ Récapitulatif:

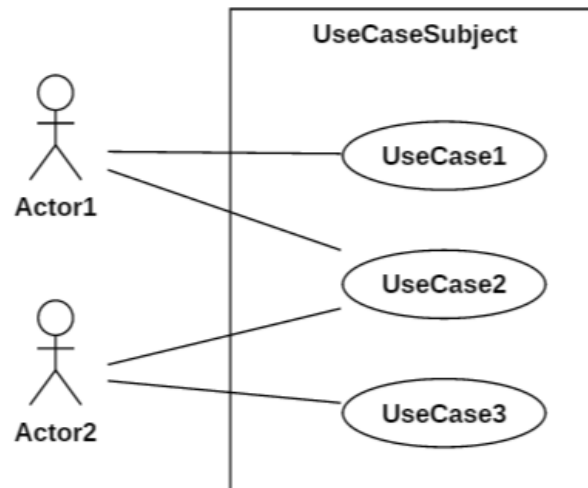


Diagramme des cas d'utilisation

Exemples:

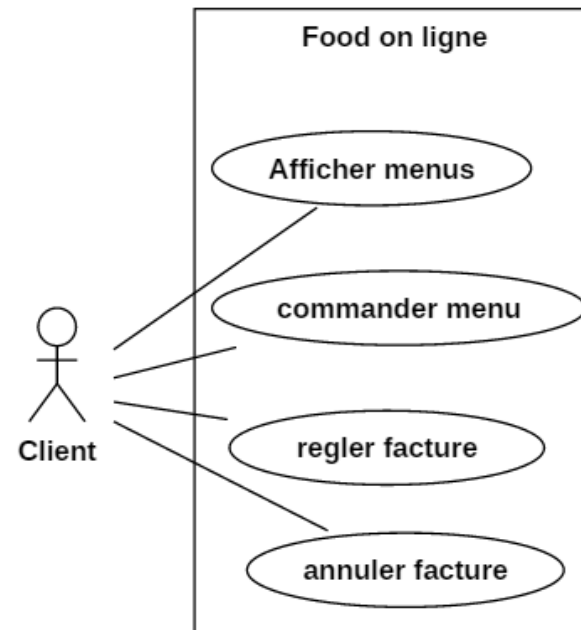
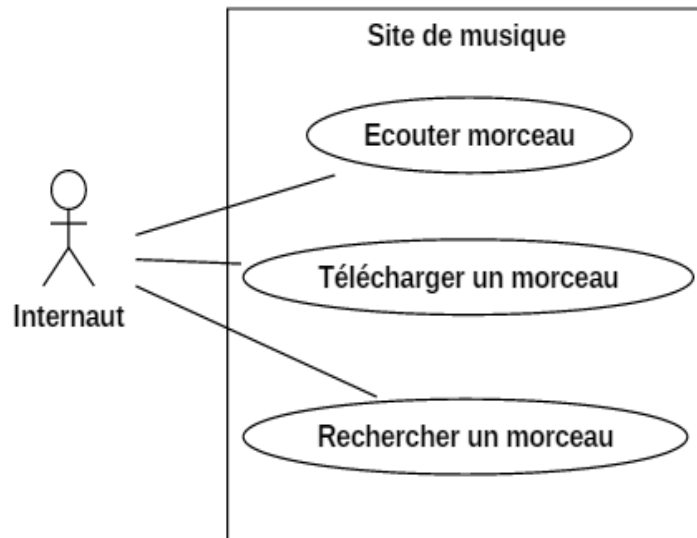


Diagramme des cas d'utilisation

■ Relations entre cas d'utilisation:

Pour clarifier un DCU, UML permet de représenter les relations et dépendances entre cas d'utilisation.

- ▷ Trois types de relations:
 - ▷ **Généralisation:** Un cas A est une généralisation d'un cas B si B est un cas particulier de A.
 - ▷ **Inclusion:** Un cas B est inclus dans un cas A si le comportement décrit par le cas B est inclus dans le comportement du cas A : on dit alors que le cas A dépend de B. Cette dépendance est symbolisée par le stéréotype « include ».
 - ▷ **Extension:** Si le comportement de B peut être étendu par le comportement de A, on dit alors que A étend B. Une extension est souvent soumise à condition. Graphiquement, la condition est exprimée sous la forme d'une note.

Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

- **Généralisation:** Un cas A est une généralisation d'un cas B si B est un cas particulier de A.
- Le cas d'utilisation enfant hérite des propriétés et du comportement du cas d'utilisation parent et peut redéfinir le comportement du parent.
- Un cas enfant doit répondre au même besoin du cas parent.
- Le cas enfant est une alternative du cas parent.



Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

- **Inclusion:** Un cas B est inclus dans un cas A si le comportement décrit par le cas B est inclus dans le comportement du cas A : on dit alors que le cas A dépend de B. Cette dépendance est symbolisée par le stéréotype « *include* ».
- Montre que le comportement du cas inclus (traitement interne) fait partie du cas incluant.
- Les inclusions permettent aussi de décomposer un cas complexe en sous-cas plus simples.
- Les inclusions permettent également de réutiliser un traitement commun (répetitif) entre plusieurs use cases, l'authentification par exemple.



Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

▷ **Inclusion:**

▷ Exemple

▷ Modifier Produit=vérifier authentification + vérifier autorisation + modifier le produit

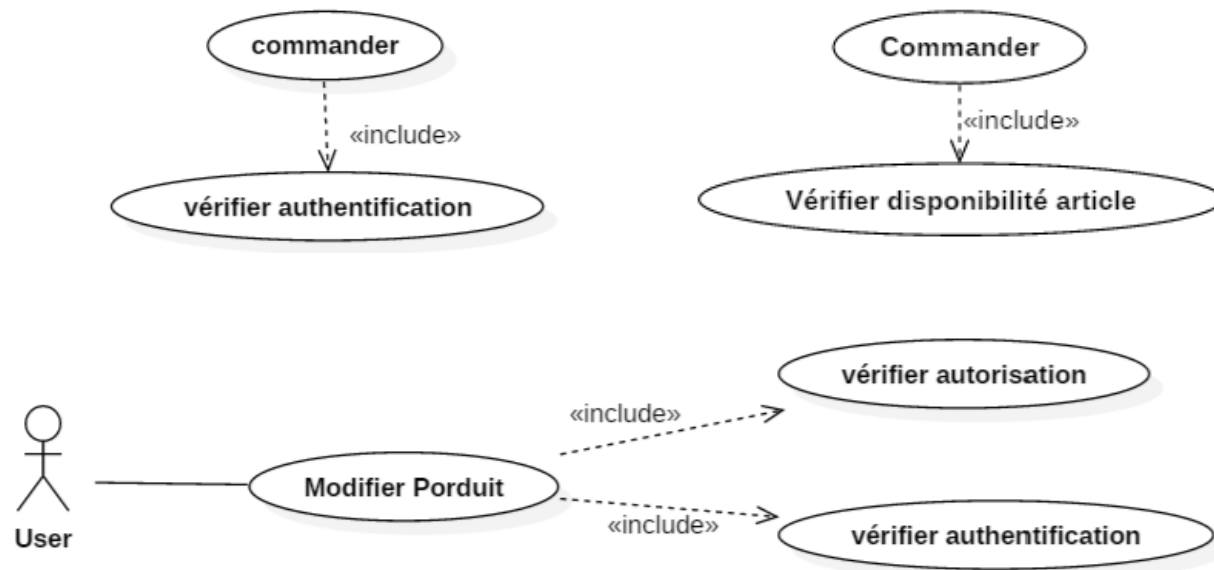


Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

- ▶ **Extension:** Si le comportement de B peut être étendu par le comportement de A, on dit alors que A étend B.
- ▶ Décrit généralement un comportement supplémentaire, optionnel.
- ▶ Le CU étendu est un cas indépendant du cas étendant. Il est significatif en soi. Le cas étendant qui est un cas facultatif, n'est pas nécessairement significatif en soi.
- ▶ Une extension est souvent soumise à condition. Graphiquement, la condition est exprimée sous la forme d'une note.

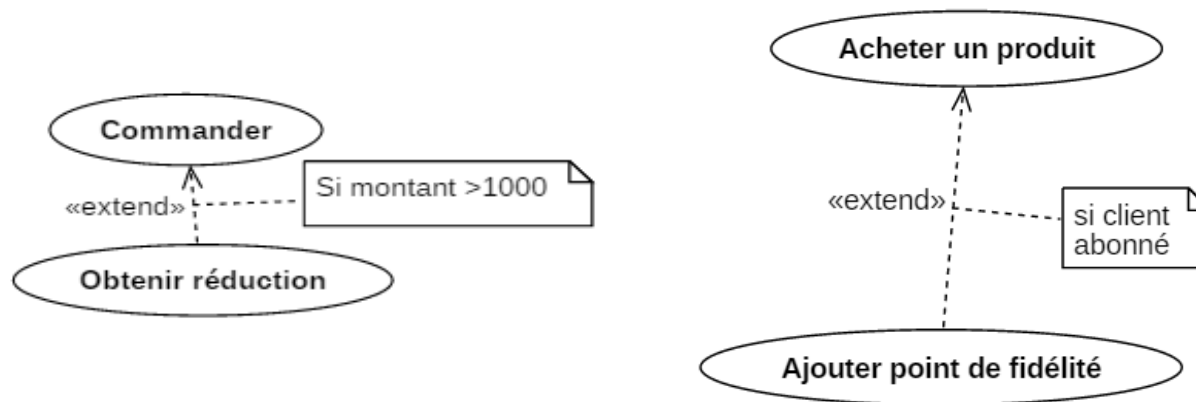


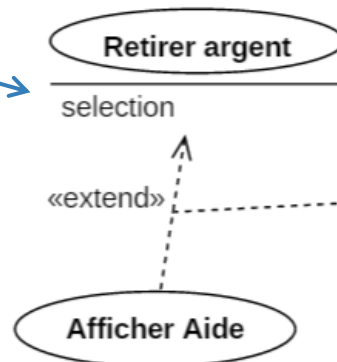
Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

► Extension:

- **Point d'extension:** représente le point de déclenchement du cas optionnel. Défini par un nom et une [description]. on peut lui associer une contrainte pour définir la condition de déclenchement du CU.
- **La condition:** définit la condition de déclenchement du CU d'extension.
- Les points d'extension et conditions, les deux sont optionnels. On peut utiliser l'un des deux, les deux ou aucun des deux.

Point d'extension



Condition

condition:{client choisit Aide}
extension point: selection

Diagramme des cas d'utilisation

Relations entre cas d'utilisation:

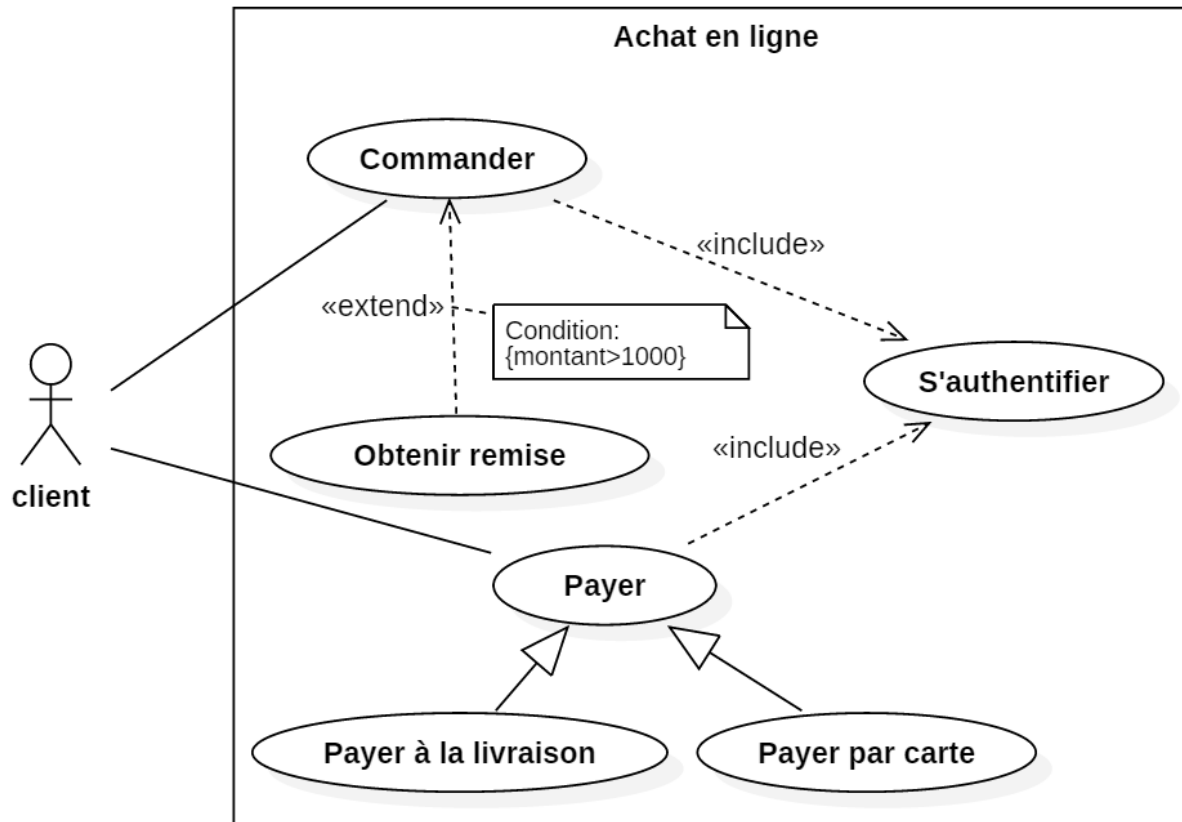


Diagramme des cas d'utilisation

■ Relations entre acteurs:

- La seule relation possible entre acteurs est la généralisation.
- Un acteur A est une généralisation d'un acteur B si l'acteur A peut être substitué par l'acteur B.
- Tous les cas d'utilisation accessibles à A le sont aussi à B, l'inverse est faux.

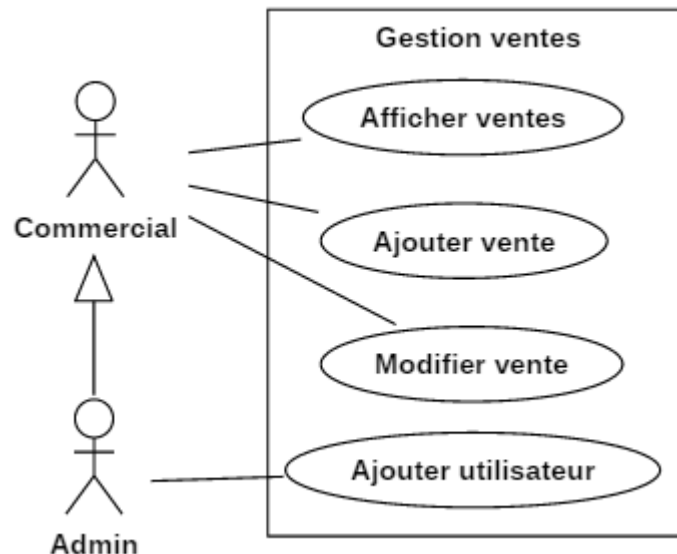


Diagramme des cas d'utilisation

■ Récapitulatif:

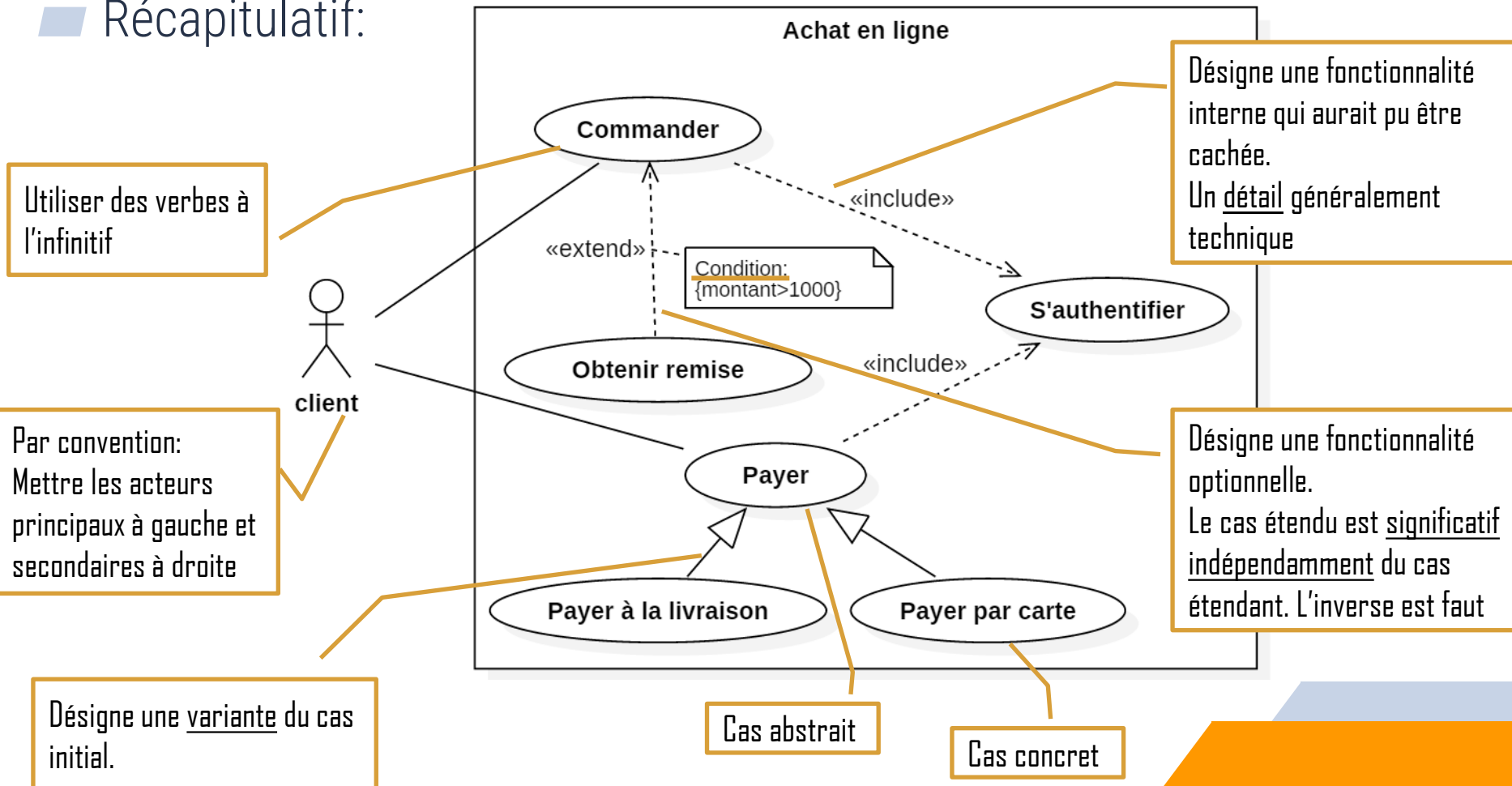


Diagramme des cas d'utilisation

■ Description textuelle des cas d'utilisation:

- ▷ Un simple nom peut être insuffisant pour décrire un cas d'utilisation.
- ▷ UML ne couvre pas l'explication des CU.
- ▷ Il est donc recommandé de faire une description textuelle de chaque CU.
- ▷ La description doit inclure:
 1. **Identifiant:** P2M11C3
 2. **Nom du cas:** Exemple: Acheter un produit.
 3. **Objectif/description:** Une description résumée permettant de comprendre l'intention principale du cas d'utilisation.
 4. **Acteurs principaux.**
 5. **Acteurs secondaires.**
 6. **Dates :** Les dates de création et de mise à jour de la description courante.
 7. **Responsables :** responsables de la définition et validation du cas.
 8. **Version :** Le numéro de version.

Diagramme des cas d'utilisation

8. **Préconditions** : décrivent dans quel état doit être le système (l'application) avant que ce cas d'utilisation puisse être déclenché. Exemple: client authentifié, Commande validée.
9. **Postconditions** : Elles décrivent l'état du système à l'issue des différents scénarii (résultat vérifiable du CU).
10. **Scénarii**: Description du fonctionnement du CU sous la forme d'une séquence de messages échangés entre les acteurs et le système. ils sont décrits sous la forme d'échange d'évènements entre l'acteur et le système.
 - **Scénario nominal**: qui se déroule quand il n'y a pas d'erreur,
 - **Scénarii alternatif**: sont des variantes du scénario nominal (cas conditionnés).
 - **Scénarii d'exception**: décrivent les cas d'erreurs.
2. **Compléments**: Rubrique optionnelle, où sont décrites par exemple des spécifications non fonctionnelles, IHM, ergonomie, etc.

Exemple

1. **Identifiant:** C328
2. **Nom du cas:** Commander un produit.
3. **Objectif/description :** l'objectif de ce CU est de permettre à un client de commander un produit à travers l'application.
4. **Acteurs principaux:** Client
5. **Acteurs secondaires:** Aucun
6. **Dates :**05/12/2020.
7. **Responsables :** Responsable achat.
8. **Version :** 1.0.
9. **Préconditions :** client authentifié.
10. **Post-conditions :** Si pas d'erreur, la commande est enregistrée avec le statut « en cours ».

Exemple

11. Scénario nominal:

- 1) Le client sélectionne la quantité du produit
- 2) Le client sélectionne la couleur
- 3) Le client ajoute le produit à son panier
- 4) Le système enregistre la commande avec le statut en cours
- 5) Le client clique sur Passer commande
- 6) Le système demande les informations de livraison
- 7) Le client sélectionne le mode de livraison
- 8) Le client saisit les informations de livraison
- 9) Le système demande les informations de paiement
- 10) Le client remplit les informations de paiement
- 11) Le système vérifie les informations de paiement
- 12) Le système affiche le récap de la commande et demande au client de la valider
- 13) Le client valide la commande.
- 14) Le système modifie le statut de la commande en validée
- 15) Le système envoie un email de confirmation au client
- 16) Le système envoie une notification au service de ventes.

12. Scénario alternatif :

A1. Si les informations de paiement sont erronées:

- Afficher un message d'erreur pour avertir le client.
- Revenir au formulaire de saisi des informations de paiement. (étape n° 5)

13. Scénario d'exception :

- E1. Si après validation, le produit n'est plus disponible avertir le client.
- Désactiver le produit dans le panier
- Remettre le statut de la commande en « à annuler »

14. **Compléments:** les interfaces client doivent être faciles à utiliser évitant toutes ambiguïtés.

Diagramme des cas d'utilisation

■ Description textuelle des cas d'utilisation:

- La description textuelle est un premier pas dans l'analyse du besoin.
- La description textuelle peut être représentée graphiquement avec un diagramme d'activité.
- Les différents scénarii identifiés servent aussi à rédiger les cas de test associés à cette fonctionnalité.

Diagramme des cas d'utilisation

■ Estimation du projet basée sur les UCP (Use Case Points) :

L'estimation basée sur les UCP prend en compte les aspects suivants:

- ▶ Complexité du cas d'utilisation : Le nombre de scénarii et étapes nécessaires à sa réalisation (classé en simple, moyen ou complexe)
- ▶ Le nombre et la complexité des acteurs (classé en simple, moyen ou complexe).
- ▶ Les facteurs techniques du cas d'utilisation telles que la concurrence, la réutilisabilité, la sécurité et la performance.
- ▶ Les facteurs environnementaux telles que l'expérience et les connaissances des équipes de développement, la qualité des outils, etc.
- ▶ Facteur de productivité: Nombre d'heures/homme nécessaire pour réaliser un UCP (vélocité). Elle est calculée en se basant sur des projets précédents.
Exemple : 20h par UCP.

Diagramme des cas d'utilisation

Exemple:

$$UCP = UUCP * TCF * ECF * PF$$

1. Unadjusted Use Case Points (UUCP)= UUCW+UAW.
2. The Technical Complexity Factor (TCF).
3. The Environment Complexity Factor (ECF)
4. Productivity Factor (PF)

Actor Type	Description	Weight	Number of Actors	Result
Simple	The actor represents another system with a defined application programming interface.	1	0	0
Average	The actor represents another system interacting through a protocol, like Transmission Control Protocol/Internet Protocol.	2	0	0
Complex	The actor is a person interacting via an interface.	3	4	12
Total UAW				12

Environmental Factor	Description	Weight	Perceived Impact	Calculated Factor (Weight*Perceived Complexity)
E1	Familiarity With UML	1.5	5	7.5
E2	Part-Time Workers	-1	0	0
E3	Analyst Capability	0.5	5	2.5
E4	Application Experience	0.5	0	0
E5	Object-Oriented Experience	1	5	5
E6	Motivation	1	5	5

Use Case Type	Description	Weight	Number of Use Cases	Result
Simple	Simple user interface. Touches only a single database entity. Its success scenario has three steps or less. Its implementation involves less than five classes.	5	7	35
Average	More interface design. Touches two or more database entities. Between four and seven steps. Its implementation involves between five and 10 classes.	10	13	130
Complex	Complex user interface or processing. Touches three or more database entities. More than seven steps. Its implementation involves more than 10 classes.	15	3	45
Total UUCW				210

Technical Factor	Description	Weight	Perceived Complexity	Calculated Factor (Weight*Perceived Complexity)
T1	Distributed System	2	1	2
T2	Performance	1	3	3
T3	End User Efficiency	1	3	3
T4	Complex Internal Processing	1	3	3
T5	Reusability	1	0	0
T6	Easy to Install	0.5	0	0
T7	Easy to Use	0.5	5	2.5
T8	Portable	2	0	0
T9	Easy to Change	1	3	3
T10	Concurrency	1	0	0
T11	Special Security Features	1	0	0
T12	Provides Direct Access for Third Parties	1	3	3
T13	Special User Training Facilities Are Required	1	0	0
Technical Total Factor				19.5

Diagramme des cas d'utilisation

■ Recensement des cas d'utilisation:

- UML est un langage qui ne fournit pas une méthode pour la réalisation de ses diagrammes.
- Pour identifier les cas d'utilisation, il faut se placer du point de vue de chaque acteur et déterminer comment et surtout pourquoi il se sert du système.
- Il faut éviter les redondances et limiter le nombre de cas en se situant à un bon niveau d'abstraction.
- Trouver le bon niveau de détail pour les cas d'utilisation est un problème difficile qui nécessite de l'expérience.
- Eviter d'introduire un séquençement des actions des utilisateurs et du système.

Diagrammes d'UML

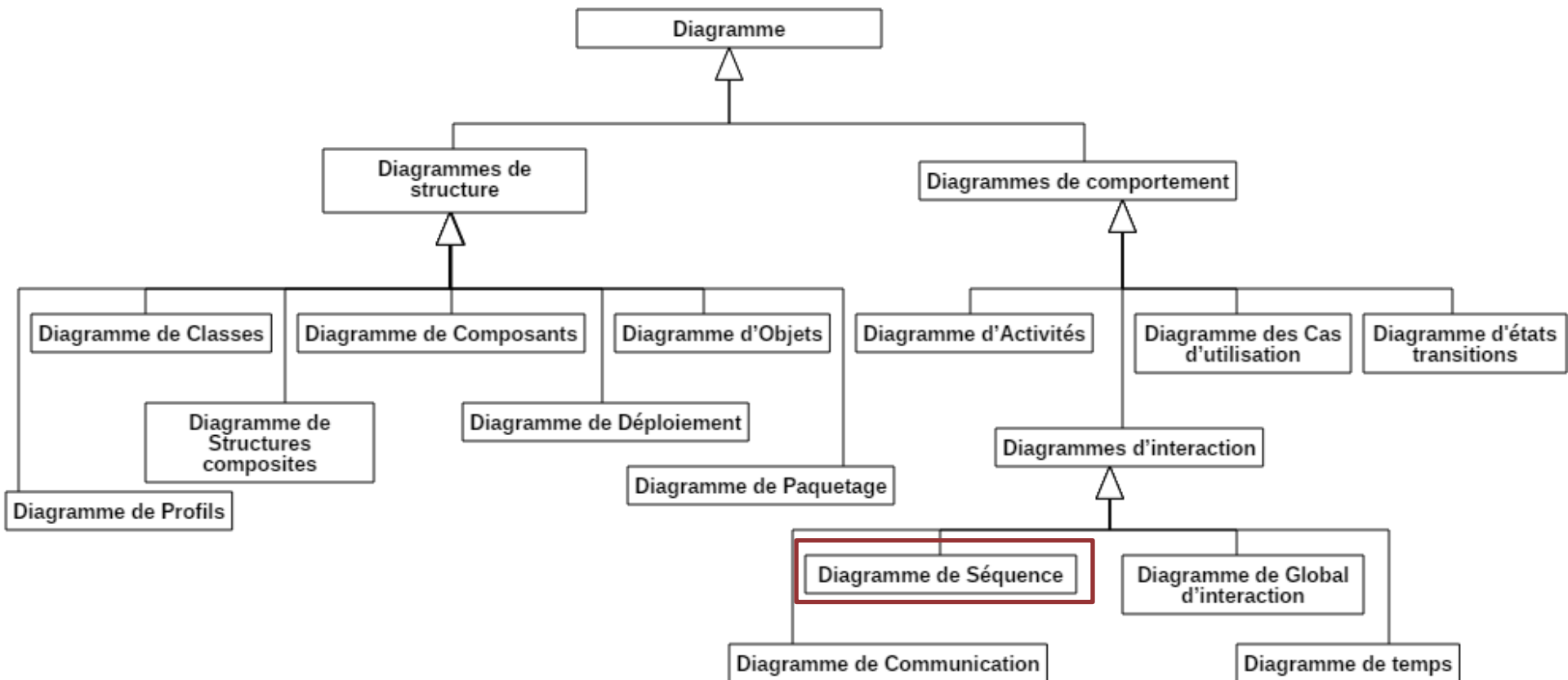


Diagramme de séquence

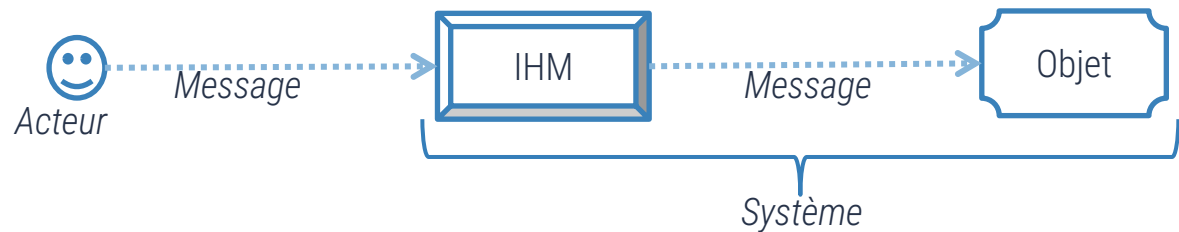
Diagramme de séquence

L'objectif du diagramme de séquence est de représenter les interactions entre objets, acteurs et système, en indiquant la chronologie des échanges.

Il est généralement utilisé pour décrire un cas d'utilisation en considérant les différents scénarios associés.

Éléments du diagramme:

- ▷ Acteurs
- ▷ Classes
- ▷ IHM/Système
- ▷ Messages



Représenté sous la forme d'un rectangle

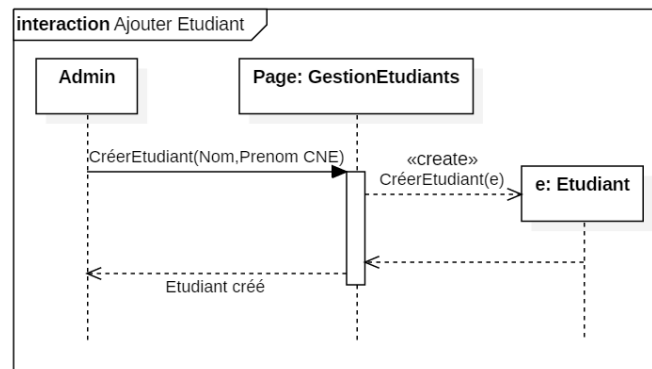


Diagramme de séquence

Ligne de vie

- ▷ Représente un participant: acteur, interface/système, objet/classe.
- ▷ Représentée par une ligne verticale en pointillés.

Période d'activation: est déclenchée suite à la réception d'un message.

- ▷ Représentée par un rectangle.
- ▷ Un objet peut être activé plusieurs fois au cours de son existence.

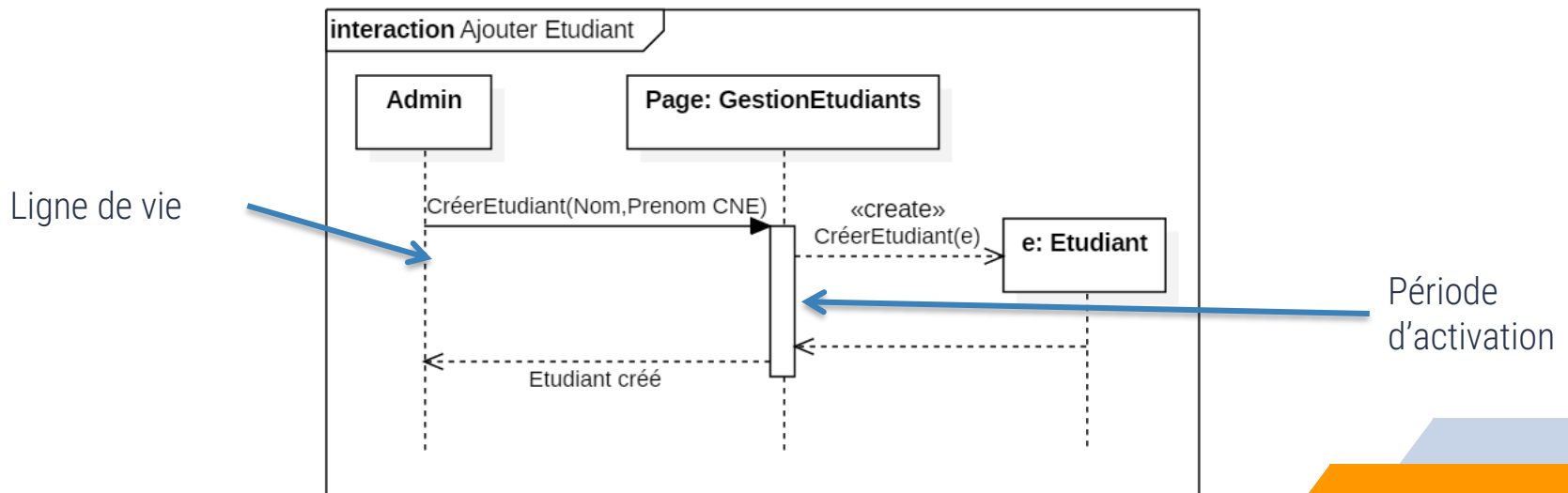


Diagramme de séquence

Message:

- ▷ Définit une communication entre un objet/acteur.
- ▷ Plusieurs types existent dont:
 - ▷ Synchrones
 - ▷ Asynchrones
 - ▷ Message de création
 - ▷ Message de suppression
 - ▷ Message de retour

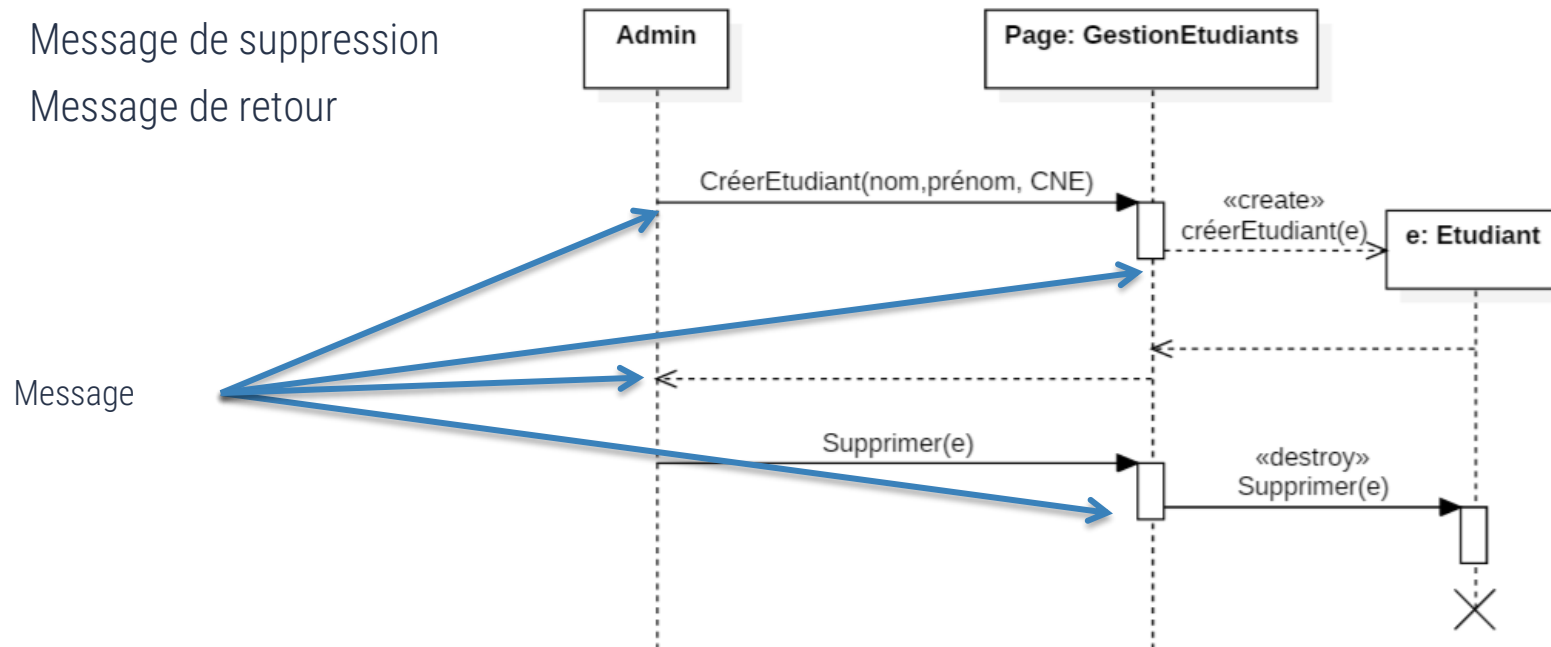


Diagramme de séquence

Message:

- ▶ **Un message synchrone:** l'émetteur reste en attente de la réponse à son message avant de poursuivre ses actions (message bloquant).
 - ▶ Exemple: Appeler une méthode et attendre sa réponse/résultat.
 - ▶ On peut associer aux messages d'appel de méthode un message de retour (en pointillés) marquant la reprise du contrôle par l'objet émetteur du message synchrone.
- ▶ **Un message asynchrone:** l'émetteur n'attend pas la réponse à son message, il poursuit l'exécution de ses opérations (message non bloquant).
 - ▶ Envoi d'un signal: déclenche une réaction chez le récepteur, de façon asynchrone et sans réponse.
 - ▶ Envoi d'une notification/ alerte (si le système de messagerie est en panne, le traitement continue sans bocalage)


Message Synchrone


Message Asynchrone


Message Retour

Diagramme de séquence

Message:

▷ Exemple:

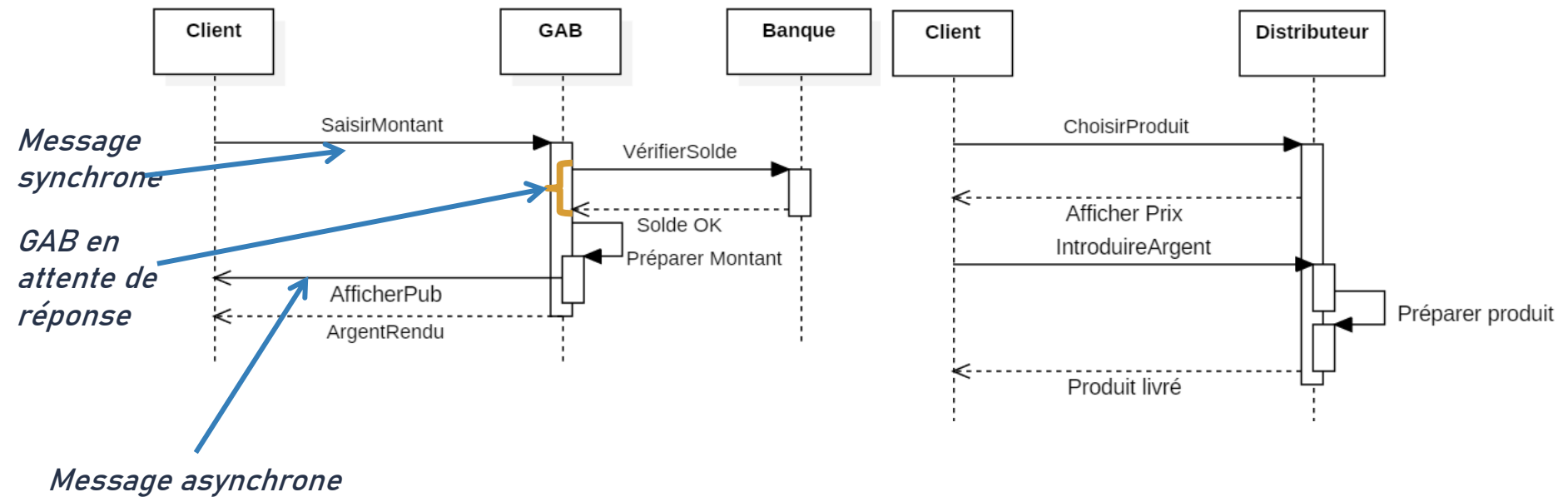


Diagramme de séquence

Message:

- **Création:** message résultant de la création d'un objet (nouvelle ligne de vie).
- **Destruction:** message de fin de ligne de vie.
- **Trouvé:** l'événement de réception est connu mais pas l'événement d'émission.
- **Perdu:** l'événement d'envoi est connu mais pas l'événement de réception.
- **Réflexif:** Appel d'une opération interne.
- **Retour:** Indique la fin d'un appel d'une méthode ou traitement, en particulier synchrone, avec ou sans valeur retournée. Il reste optionnel pour éviter d'encombrer le diagramme.

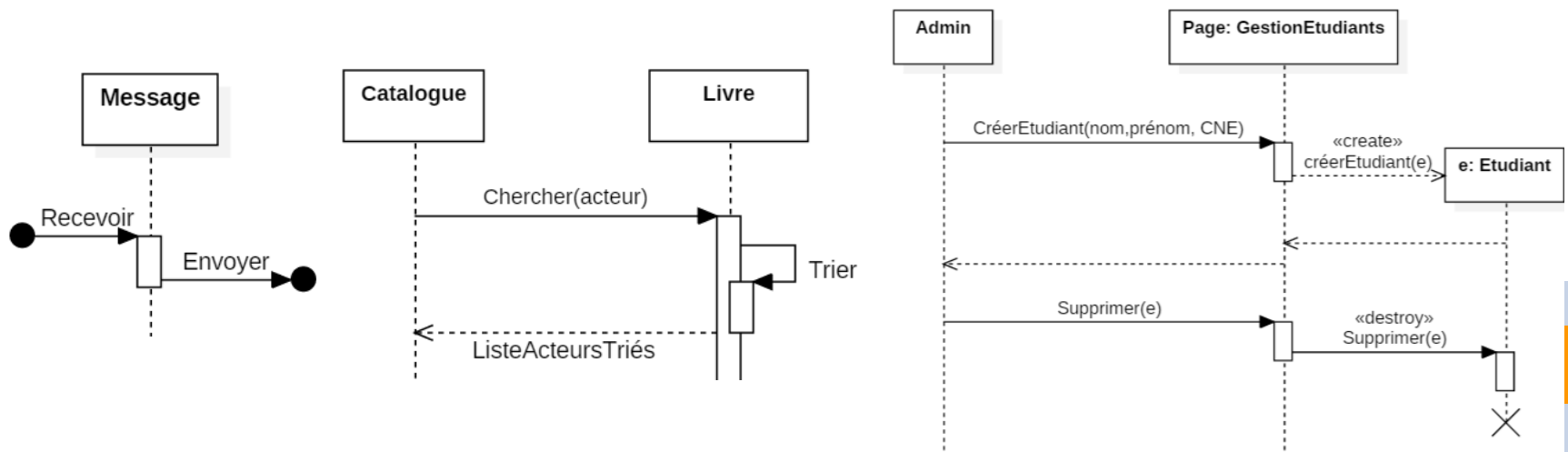


Diagramme de séquence

Exemple:

- ▶ Comment se propage les virus ?

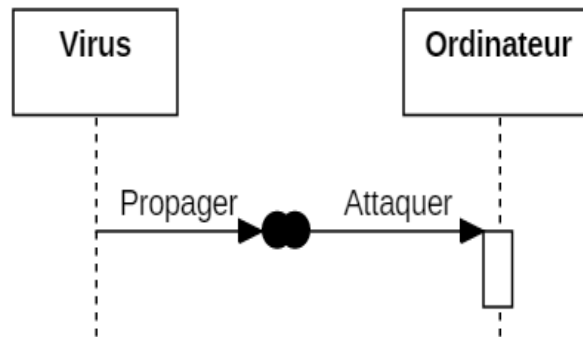


Diagramme de séquence

■ Fragment combiné:

- ▷ Dans un diagramme de séquence, il est possible de distinguer des sous-ensembles d'interactions qui constituent des fragments.
- ▷ Représente un groupe de messages conditionnels ou répétitifs.
- ▷ Un fragment combiné est utilisé pour décrire les comportements complexes qui ne peuvent pas être représentés avec les éléments de base.
- ▷ Un fragment combiné peut contenir des conditions et des boucles, telles que les branchements « if », « else », « while », « not », etc.
- ▷ Les fragments combinés peuvent être imbriqués les uns dans les autres pour représenter des comportements complexes et des séquences d'actions.

Diagramme de séquence

■ Fragment combiné:

- Un fragment d'interaction se représente globalement comme un diagramme de séquence dans un rectangle avec indication dans le coin à gauche du nom du fragment.
- Treize opérateurs ont été définis dans UML : alt, opt, loop, par, strict/weak, break, ignore/consider, critical, negative, assertion et ref.

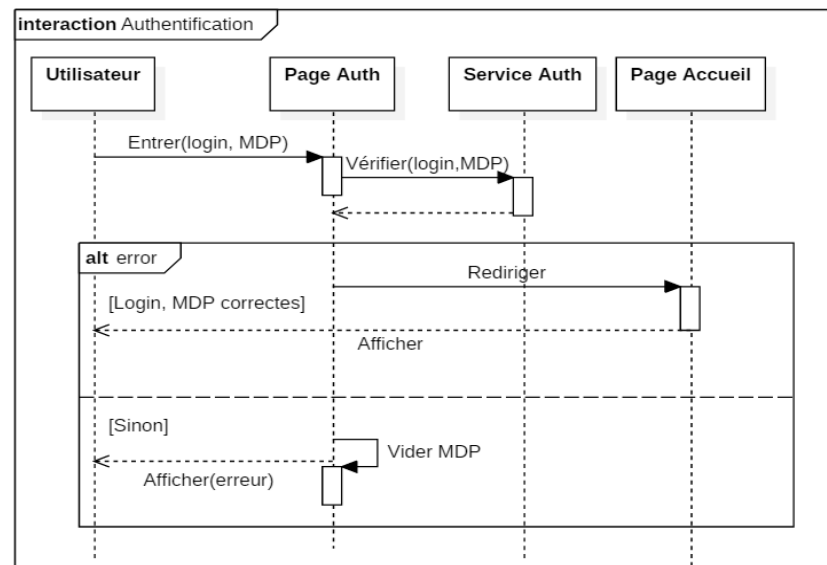


Diagramme de séquence

Opérateur alt : L'opérateur alt correspond à une instruction de test avec une ou plusieurs alternatives possibles. Il est aussi permis d'utiliser les clauses de type sinon.

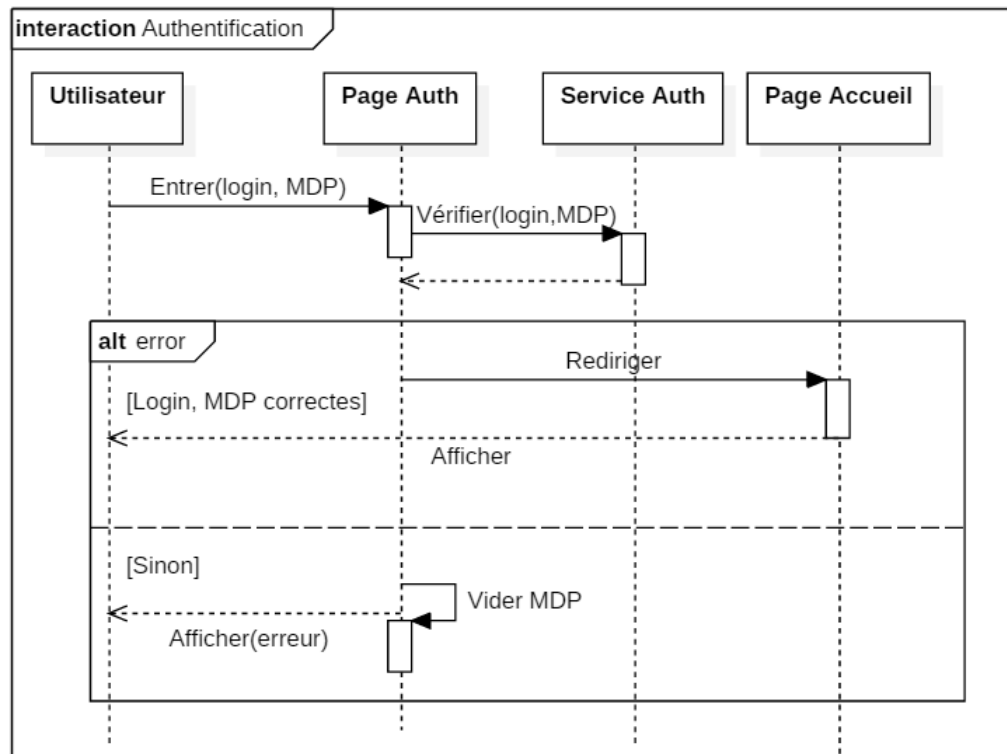


Diagramme de séquence

Exemple:

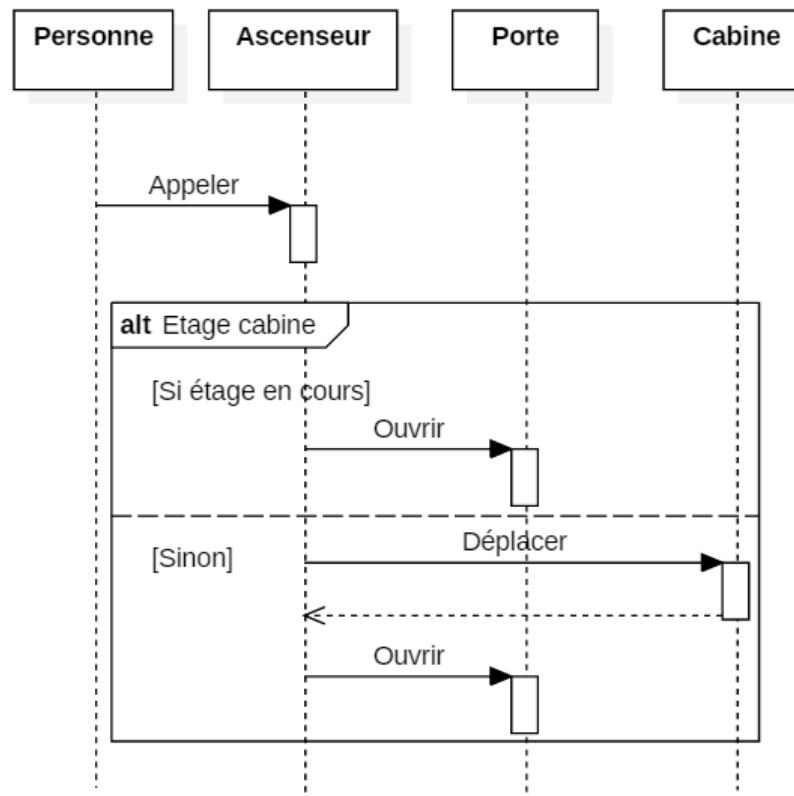


Diagramme de séquence

■ **Opérateur opt:** L'opérateur opt (optional) correspond à une instruction de test sans alternative (sinon).

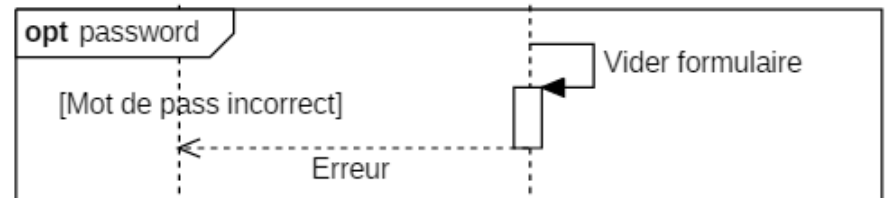
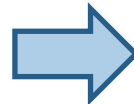
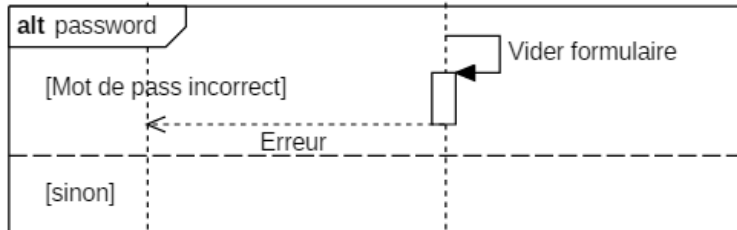


Diagramme de séquence

■ **Opérateur loop:** L'opérateur loop correspond à une instruction de boucle qui permet d'exécuter une séquence d'interactions tant qu'une condition est satisfaite.

▷ La syntaxe d'une boucle est la suivante :

loop (min, max) [guard] (min et max deux entiers tel que $\text{min} \geq 0$ et $\text{max} \geq \text{min}$).

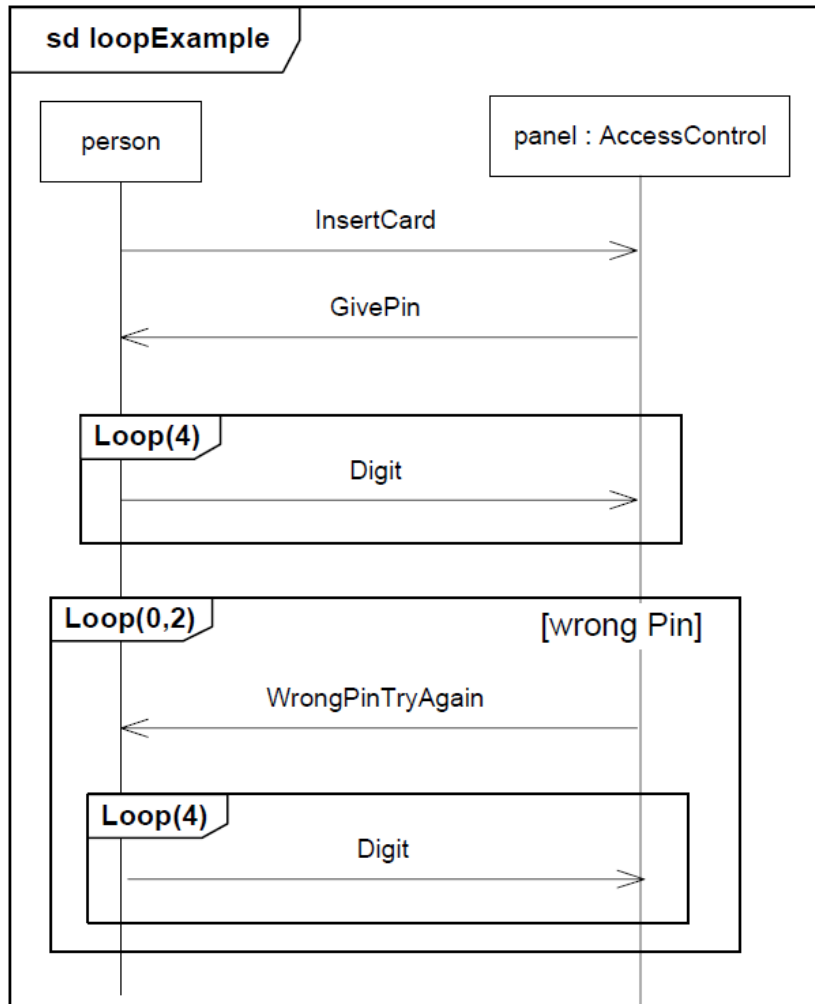
où la boucle est répétée au moins min fois avant qu'une éventuelle condition booléenne ne soit testée (la condition est placée entre crochets sur la ligne de vie) ; tant que la condition est vraie, la boucle continue, au plus max fois.

loop(valeur) est équivalent à loop(valeur, valeur).

loop est équivalent à loop(0, *), où * signifie « illimité ».

Diagramme de séquence

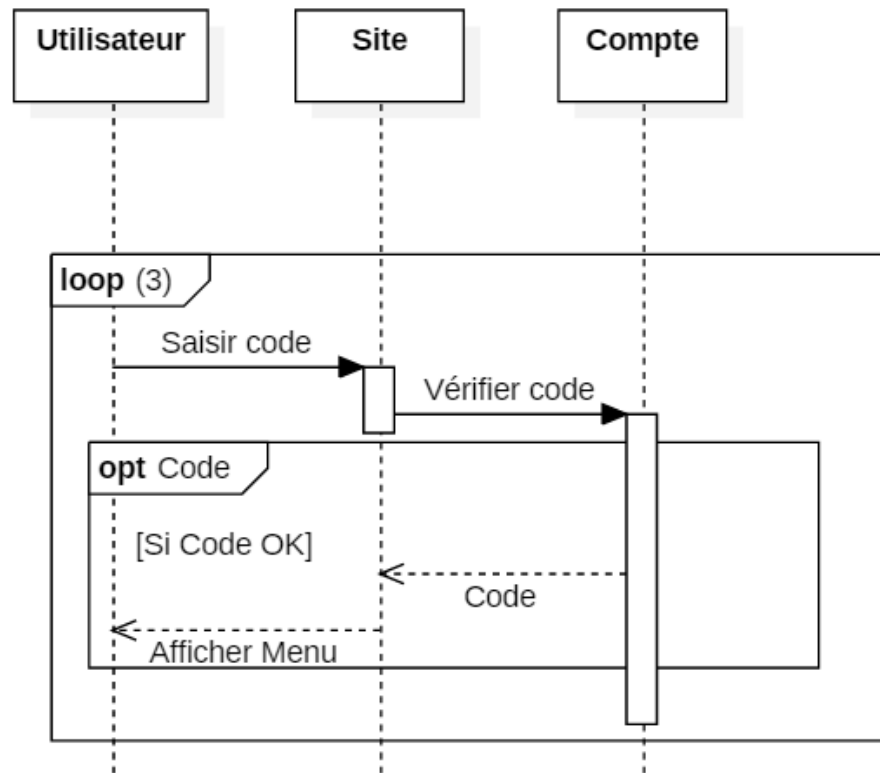
Exemple:



[OMG]

Diagramme de séquence

Exemple:



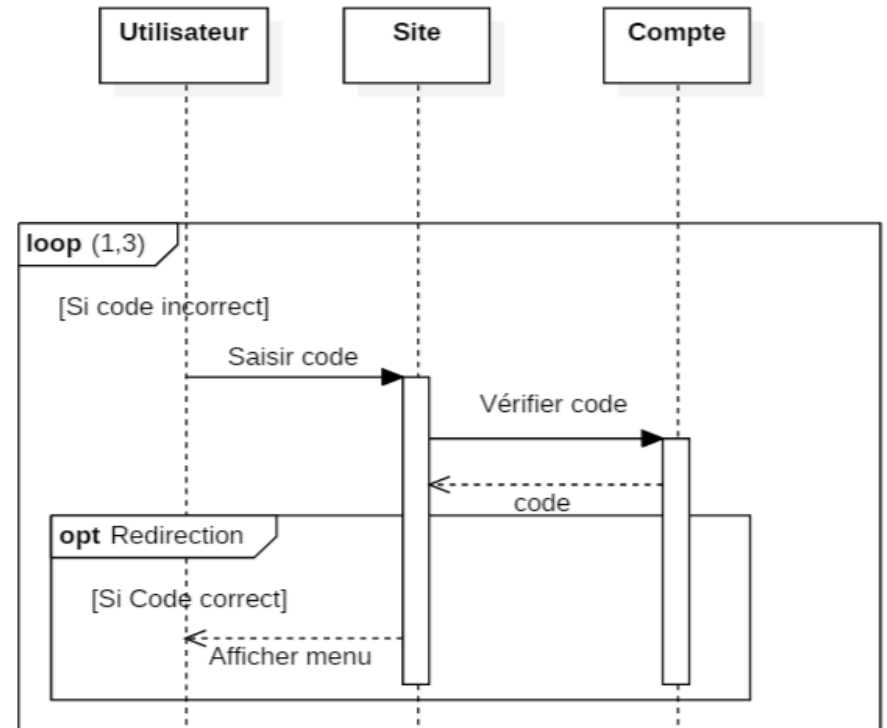
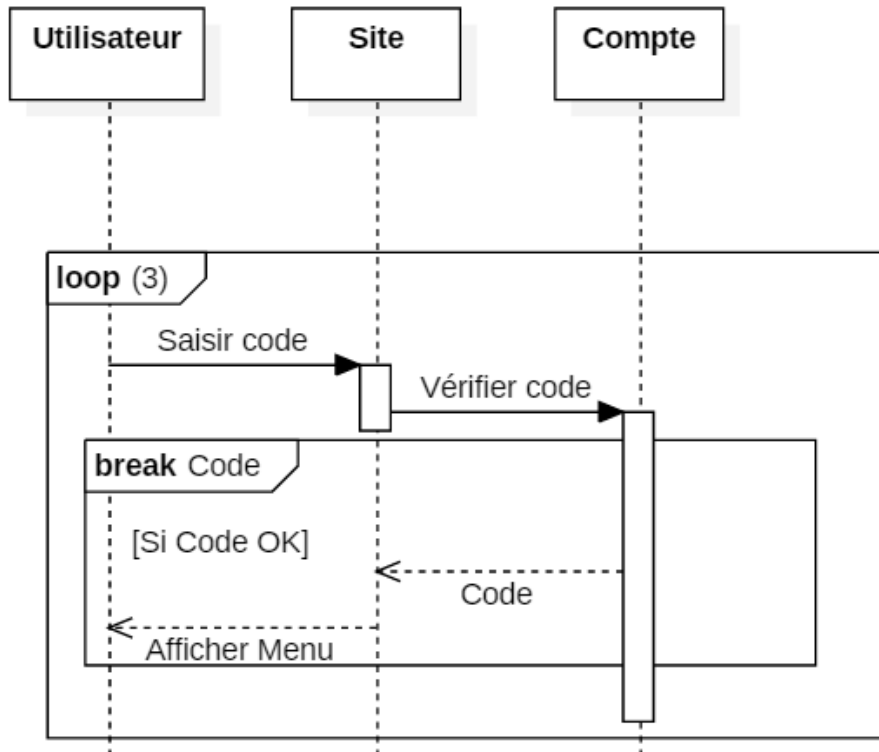
L'utilisateur devra ressaisir son code trois fois, même s'il est correct.

Diagramme de séquence

- **Opérateur *break*:** L'opérateur break permet de représenter une situation exceptionnelle correspondant à un scénario de rupture par rapport au scénario général. Le scénario de rupture s'exécute si la condition de garde est satisfaite.
- Les messages au sein du fragment break sont exécutés avant de sortir.
- L'opérateur break doit toujours couvrir toutes les lignes de vie du fragment concerné.

Diagramme de séquence

Exemple:

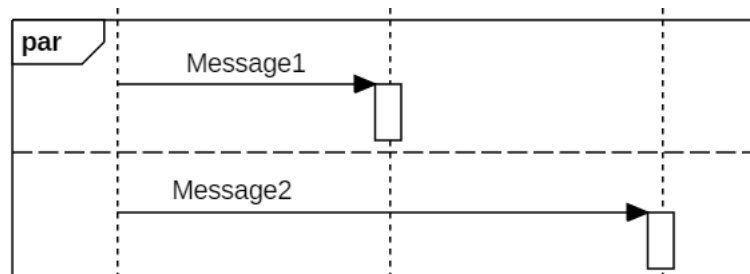


Sortir une fois le code saisi est correcte.

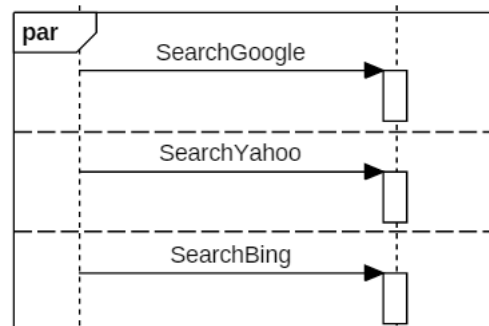
Diagramme de séquence

Fragment combiné:

- *Opérateur par*: L'opérateur par (parallel) permet de représenter deux séries d'interactions qui se déroulent en parallèle.



Rechercher sur google, bing ou yahoo dans n'importe quel ordre, même en parallèle



NB:
L'ordre à l'intérieur des opérandes doit être maintenu.

Diagramme de séquence

Fragment combiné:

- L'*opérateur strict* est utilisé quand l'ordre d'exécution des opérations doit être strictement respecté. Ce fragment est utile surtout lorsque deux parties d'un diagramme n'ont pas de ligne de vie en commun.

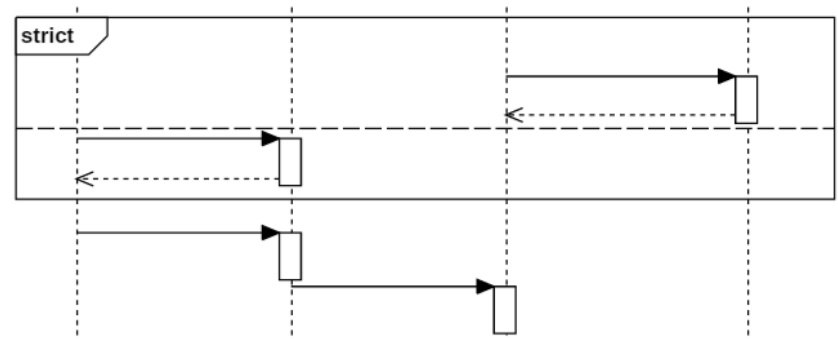
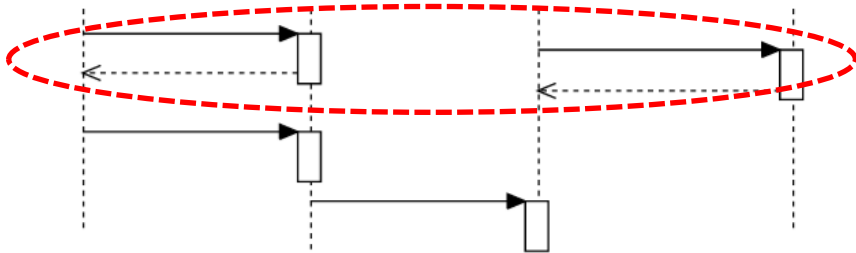


Diagramme de séquence

Fragment combiné:

- ▶ L'opérateur **seq** (Weak Sequencing) représente un ordre d'exécution faible. Tel que:
 - ▶ Si des interactions, dans des opérandes différents, font intervenir **différentes** lignes de vie (disjointes) le weak seq est équivalent au fragment **par**.
 - ▶ Si les opérations, dans des opérandes différents, se font sur la **même** ligne de vie, le weak seq est équivalent à un **strict** seq.
 - ▶ L'ordre des interactions dans chacun des opérandes est conservé au final.

Recherche **sur** Google peut se faire en parallèle avec Bing et Yahoo.
Mais recherche Bing avant Yahoo.

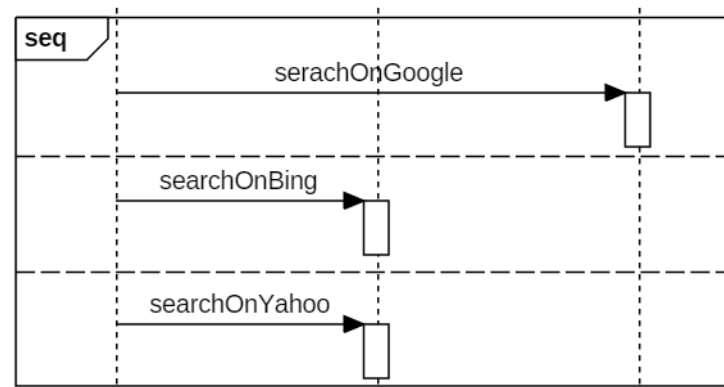
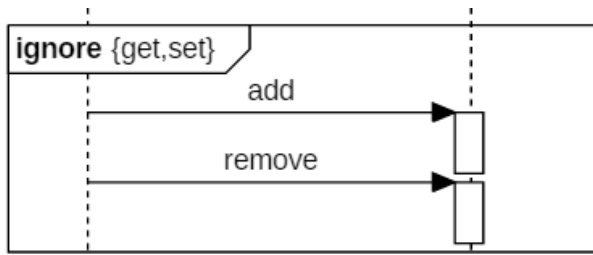


Diagramme de séquence

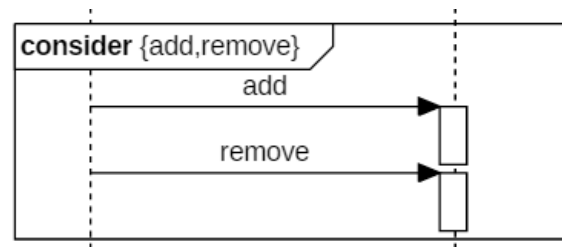
Fragment combiné:

- **Opérateurs ignore et consider:** Les opérateurs **ignore** et **consider** sont utilisés pour des fragments d'interactions dans lesquels on veut montrer que certains messages peuvent être soit absents sans avoir d'incidence sur le déroulement des interactions (ignore), des tests par exemple, soit obligatoirement présents (consider).
- L'utilisation de consider, revient à définir tous les autres messages comme à ignorer.

('ignore' | 'consider') '{ <message-name> [, ' <message-name>]* }'



Ignore get() and set() messages, if any.



Consider only add() or remove() messages, ignore any other.

Diagramme de séquence

Fragment combiné:

- **Opérateur critical:** L'opérateur critical permet d'indiquer qu'une séquence d'interactions ne peut être interrompue compte tenu du caractère critique des opérations traitées. On considère que le traitement des interactions comprises dans la séquence critique est atomique.
- **L'opérateur neg** (negative) permet d'indiquer qu'une séquence d'interactions est invalide.

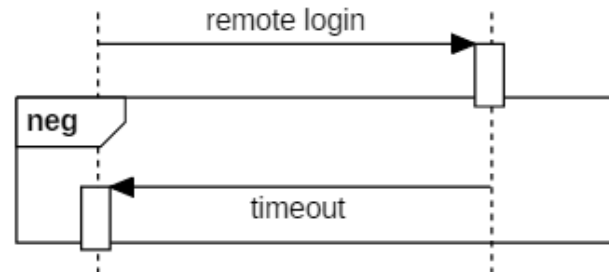
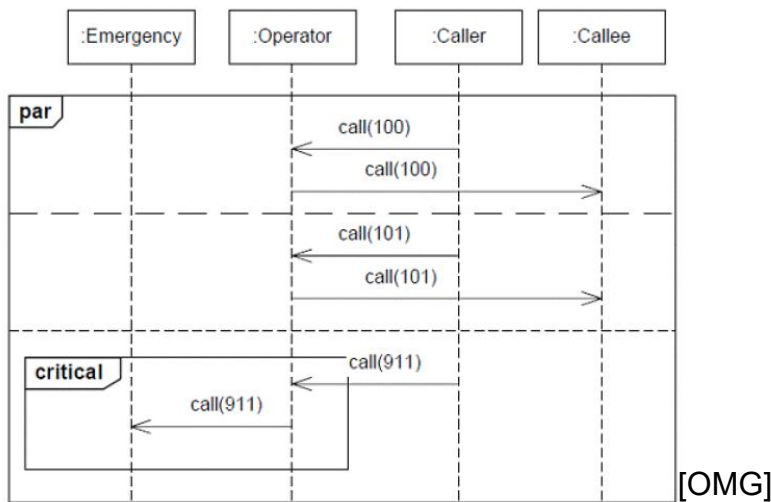


Diagramme de séquence

Fragment combiné:

- *L'opérateur assert (assertion)*: permet d'indiquer qu'une séquence d'interactions est l'unique séquence possible (confirmation). Toute autre alternative de message est invalide.

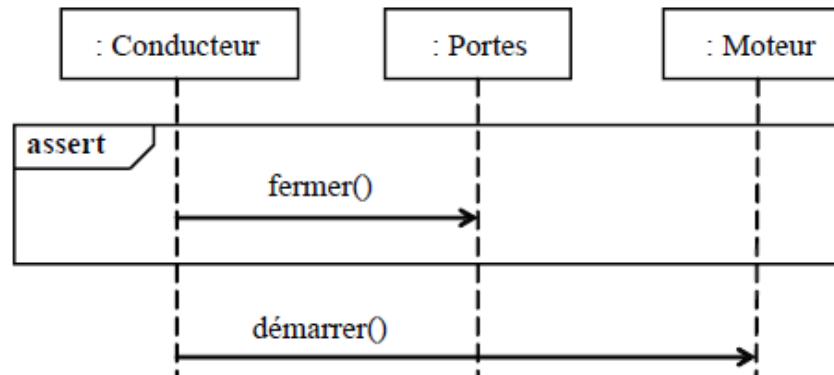


Diagramme de séquence

Interaction use ref:

- L'opérateur *ref* : permet d'appeler une séquence d'interactions (ou un autre diagramme de séquence) décrite ailleurs, constituant ainsi une sorte de sous-diagramme de séquence.

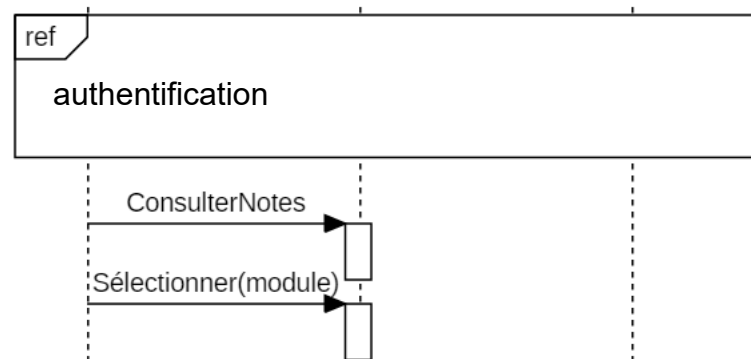


Diagramme de séquence

Continuation:

- Une continuation est une manière syntaxique de définir des continuations de différentes branches d'un fragment combiné de type alt et seq.
- Est une sorte de raccourcis vers un autre diagramme pour l'enrichir.

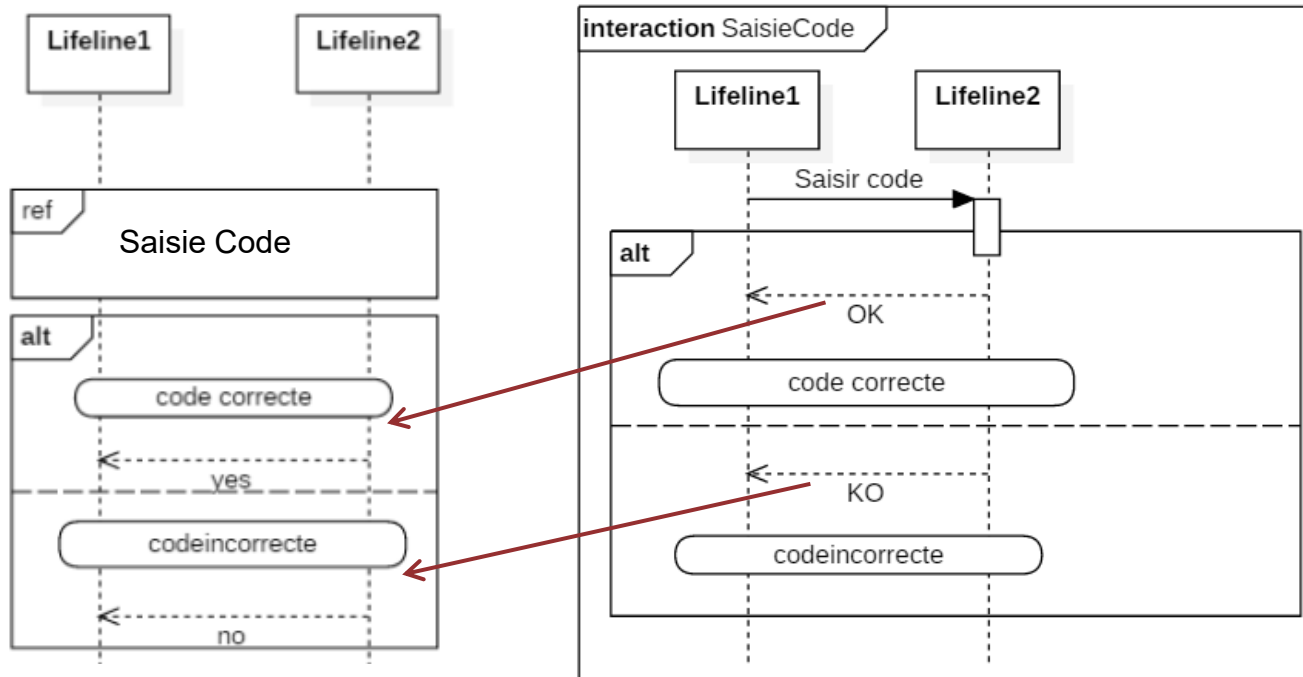


Diagramme de séquence

■ Exemple:

- ▶ Construisez un diagramme de séquence ayant trois lignes de vie : Conducteur, Train et Passager.

Comment interdire aux passagers d'ouvrir les portes quand le train a démarré ?

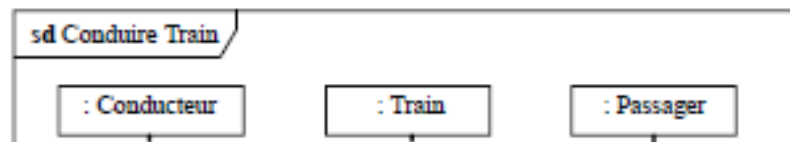


Diagramme de séquence

Exemple:

- Dessinez un diagramme de séquence qui présente trois objets : un robot + deux capteurs (une caméra et un détecteur de chocs). Le diagramme doit contenir un fragment d'interaction qui montre que les capteurs fonctionnent en même temps (ils peuvent envoyer à tout moment des messages au robot).

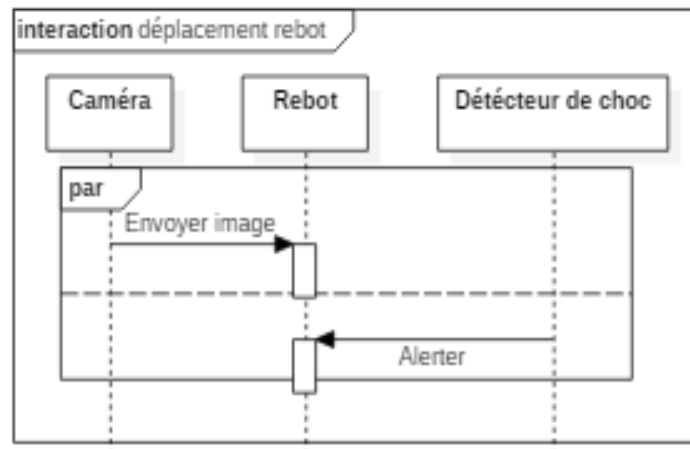
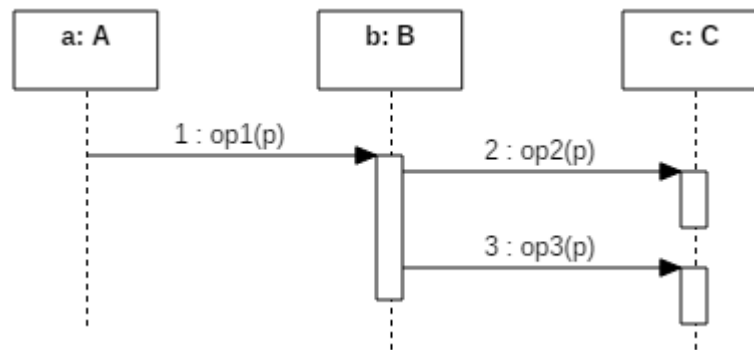


Diagramme de séquence

■ Diagramme de séquence boîte blanche (à suivre...):

▷ *Implémentation:*



```
class B {  
    C c;  
    op1 (p: Type ){  
        c. op2 (p);  
        c. op3 (p);  
    }  
}
```

```
class C {  
    op2 (p: Type ){  
        ...  
    }  
    op3 (p: Type ){  
        ...  
    }  
}
```

Diagramme de séquence

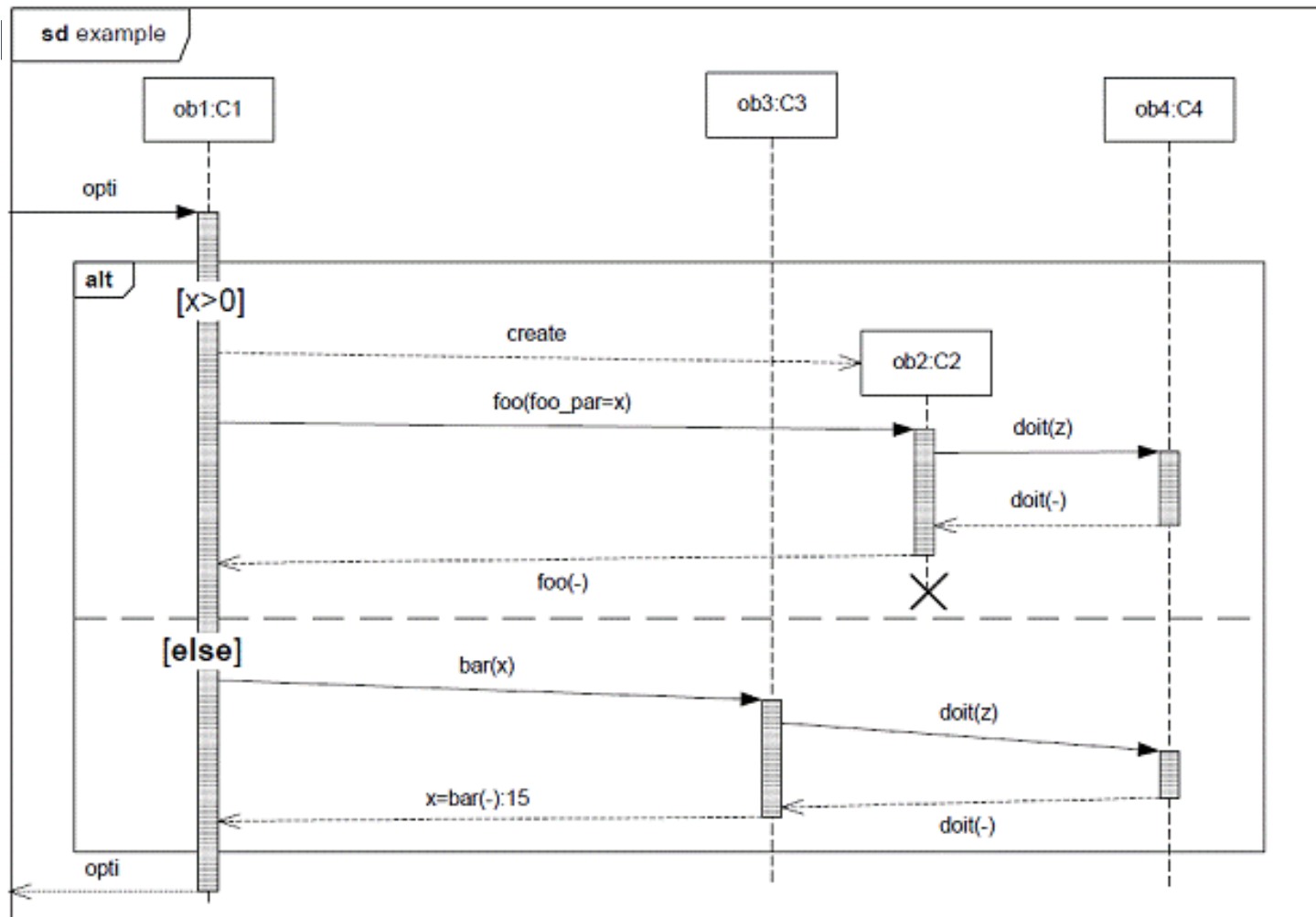


Diagramme de séquence

■ Diagramme de séquence boîte blanche (à suivre...):

► Implémentation:

```
class C1 {  
  opti () {  
    If x>0 then  
      obj2 C2=new C2();  
      obj2. foo (foo_par=x);  
    Else  
      Obj3.bar(x);  
  }  
  ... }  
}
```

```
class C2 {  
  foo (foo_par){  
    obj4.doit(z)  
  }  
  ... }  
}
```

```
class C4 {  
  doit (z){  
    ...  
  }  
}
```

```
class C3 {  
  bar (x){  
    obj4.doit(z)  
  }  
  ... }  
}
```

